PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE), PUNE

Department of Information Technology

S.Y. B.Tech Information Technology | Academic Year 2026–27

SMART CAMPUS INFORMATION AND RESOURCE MANAGEMENT SYSTEM

COMPLETE SOURCE CODE COMPENDIUM

Course: Data Structures and Algorithms (Course Code: IT204)
Standard: Modern C++17 (Zero External Library Dependencies)
Architecture: Modular Multi-File Header & Controller Hierarchy
Total Implementation Modules: 9 Integrated Subsystems

Source Code Index & File Manifest

File Path	Subsystem / Module Description	Lines of Code
`main.cpp`	Master Application Coordinator & Main Entry Point	161 lines
`src/Common.hpp`	Shared Student Model, ANSI UI Themes & Input Guards	126 lines
`src/Module1_StudentSearch.hpp`	Module 1: Student Information Management & Searching	358 lines
`src/Module2_SortingFiles.hpp`	Module 2: Student Record Sorting & External File Handling	313 lines
`src/Module3_SparseMatrix.hpp`	Module 3: Sparse Matrix Representation & Triplet Grade Analysis	341 lines
`src/Module4_LinkedLists.hpp`	Module 4: Dynamic Campus Services via Linked Lists (SLL, DLL, CLL)	679 lines
`src/Module5_StacksQueues.hpp`	Module 5: Reversible Operations, Expression Conversion & Queues	648 lines
`src/Module6_Trees.hpp`	Module 6: Hierarchical Indexing via Trees (BST, AVL, B+ Tree)	931 lines
`src/Module7_CampusGraph.hpp`	Module 7: Campus Network Routing (Graph Matrix/List, Prim, Dijkstra)	472 lines
`src/Module8_HashTable.hpp`	Module 8: Constant-Time Retrieval via Open Addressing Hashing	300 lines
`src/Module9_Complexity.hpp`	Module 9: Master Complexity Analysis & Theoretical Performance	80 lines
`grades.txt`	External Raw Input Grade Data File	15 lines

main.cpp — Master Application Coordinator & Main Entry Point

```cpp
/**
 * ============================================================================
 * PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE), PUNE
 * Department of Information Technology
 * S.Y. B.Tech Information Technology | Academic Year 2026-27
 * Course: Data Structures and Algorithms
 *
 * INTEGRATED ASSIGNMENT:
 * SMART CAMPUS INFORMATION AND RESOURCE MANAGEMENT SYSTEM
 * ============================================================================
 *
 * Modules Implemented:
 *  - Module 1: Student Information Management & Searching (Linear & Binary Search)
 *  - Module 2: Student Record Sorting & File Handling (Bubble, Selection, Insertion Sort)
 *  - Module 3: Sparse Matrix Representation & Grade Analysis (Triplet Form)
 *  - Module 4: Campus Services Using Linked Lists (Singly, Doubly, Circular Lists)
 *  - Module 5: Stacks, Queues & Expression Conversion (Undo Stack, Infix/Postfix/Prefix, Queues)
 *  - Module 6: Hierarchical Indexing Using Trees (BST, AVL Tree, B+ Tree)
 *  - Module 7: Campus Network Optimization (Adjacency Matrix/List, BFS, DFS, Prim MST, Dijkstra)
 *  - Module 8: Fast Record Retrieval Using Hashing (Open Addressing, Linear Probing, Tombstones)
 *  - Module 9: System Integration, Validation & Comprehensive Complexity Analysis
 * ============================================================================
 */

#include "src/Common.hpp"
#include "src/Module1_StudentSearch.hpp"
#include "src/Module2_SortingFiles.hpp"
#include "src/Module3_SparseMatrix.hpp"
#include "src/Module4_LinkedLists.hpp"
#include "src/Module5_StacksQueues.hpp"
#include "src/Module6_Trees.hpp"
#include "src/Module7_CampusGraph.hpp"
#include "src/Module8_HashTable.hpp"
#include "src/Module9_Complexity.hpp"

// Global System Coordinator
class SmartCampusSystem {
private:
    StudentManager studentMgr;
    SparseGradeMatrix sparseMatrix;
    CampusServicesManager servicesMgr;
    StackQueueManager stackQueueMgr;
    TreeIndexManager treeMgr;
    CampusNetwork campusMap;
    StudentHashTable studentHash;

public:
    SmartCampusSystem() : studentHash(11) {
        loadInitialCampusData();
    }

    // Populate initial rich demo dataset across all integrated modules
    void loadInitialCampusData() {
        // 1. Initial Students
        std::vector<Student> initialStudents = {
            Student(101, "Aarav Sharma",   "A", "09876543210", {85, 78, 92, 88}),
            Student(102, "Diya Patel",     "A", "09123456789", {92, 95, 89, 94}),
            Student(103, "Rohan Kulkarni", "B", "09822334455", {74, 82, 68, 79}),
            Student(104, "Sneha Deshmukh", "B", "09765432101", {88, 90, 85, 91}),
            Student(105, "Ishaan Joshi",   "A", "094455667778", {65, 70, 75, 72}),
            Student(106, "Ananya Roy",     "B", "091122334445", {95, 98, 94, 96})
        };

        for (const auto& s : initialStudents) {
            studentMgr.addStudent(s);
            // Record initial insertions in Undo stack
            stackQueueMgr.getUndoStack().push(ACTION_INSERT, s);
        }

        // Sync to Trees and Hash Table
        treeMgr.populateTrees(studentMgr.getStudents());
        studentHash.populateFromVector(studentMgr.getStudents());
    }

    void run() {
        while (true) {
            std::cout << "\n" << Color::CYAN << Color::BOLD
                      << "═══════════════════════════════════════════════════════════════════════════════\n"
                      << "                   PIMPRI CHINCHWAD COLLEGE OF ENGINEERING, PUNE               \n"
                      << "                        DEPARTMENT OF INFORMATION TECHNOLOGY                   \n"
                      << "           DATA STRUCTURES & ALGORITHMS | SMART CAMPUS MANAGEMENT SYSTEM       \n"
                      << "═══════════════════════════════════════════════════════════════════════════════\n"
                      << Color::RESET
                      << " [MAIN SYSTEM NAVIGATION MENU]\n"
                      << "  1. Student Information Management & Searching      (Linear & Binary Search)\n"
                      << "  2. Student Record Sorting & External File Handling (Bubble, Selection, Insertion Sort)\n"
                      << "  3. Sparse Matrix Representation & Grade Analysis   (Triplet 3-Tuple Analysis)\n"
                      << "  4. Campus Services Management Using Linked Lists   (Singly, Doubly, Circular Lists)\n"
                      << "  5. Stacks, Queues & Expression Conversion          (Undo Stack, Polish Notation, Queues)\n"
                      << "  6. Hierarchical Indexing Using Trees               (BST, AVL Tree, B+ Tree)\n"
                      << "  7. Campus Network Representation & Optimization    (Graph Matrix/List, BFS, DFS, MST, Dijkstra)\n"
                      << "  8. Student Record Retrieval Using Hashing          (Open Addressing, Linear Probing)\n"
                      << "  9. Master Complexity Analysis & Theoretical Report (Time & Auxiliary Space Tables)\n"
                      << " 10. Reload / Reset Master Campus Demo Dataset\n"
                      << "  0. Exit Smart Campus System\n"
                      << "──────────────────────────────────────────────────────────────────────────────\n";

            int choice = InputHelper::getInteger(" Select a module to launch (0-10): ");

            if (choice == 0) {
                std::cout << "\n" << Color::GREEN << Color::BOLD
                          << " [!] Thank you for using the PCCOE Smart Campus Management System. Terminating cleanly.\n"
                          << Color::RESET;
                break;
            }

```

```
            switch (choice) {
                case 1:
                    studentMgr.runMenu();
                    // Sync updated dataset to trees and hash table
                    treeMgr.populateTrees(studentMgr.getStudents());
                    break;
                case 2:
                    SortingAndFileManager::runMenu(studentMgr.getStudents());
                    break;
                case 3:
                    sparseMatrix.runMenu();
                    break;
                case 4:
                    servicesMgr.runMenu();
                    break;
                case 5:
                    stackQueueMgr.runMenu(studentMgr.getStudents());
                    break;
                case 6:
                    treeMgr.runMenu(studentMgr.getStudents());
                    break;
                case 7:
                    campusMap.runMenu();
                    break;
                case 8:
                    studentHash.runMenu(studentMgr.getStudents());
                    break;
                case 9:
                    ComplexityMaster::displayFullAnalysis();
                    InputHelper::pause();
                    break;
                case 10:
                    studentMgr.getStudents().clear();
                    loadInitialCampusData();
                    std::cout << Color::GREEN << " [Success] Campus system data reloaded to baseline demo state.\n" << Color::RESET;
                    InputHelper::pause();
                    break;
                default:
                    std::cout << Color::RED << " Invalid selection! Please enter a number between 0 and 10.\n" << Color::RESET;
                    InputHelper::pause();
            }
        }
    }
};

int main() {
    // Set standard I/O synchronization
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    SmartCampusSystem campusSystem;
    campusSystem.run();

    return 0;
}
```

src/Common.hpp

Shared Student Model, ANSI UI Themes & Input Guards

```cpp
#ifndef COMMON_HPP
#define COMMON_HPP

#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <limits>
#include <sstream>
#include <cmath>

// ============================================================================
// ANSI Color Escape Sequences for clear and pleasant terminal presentation
// ============================================================================
namespace Color {
    const std::string RESET   = "\033[0m";
    const std::string BOLD    = "\033[1m";
    const std::string RED     = "\033[31m";
    const std::string GREEN   = "\033[32m";
    const std::string YELLOW  = "\033[33m";
    const std::string BLUE    = "\033[34m";
    const std::string MAGENTA = "\033[35m";
    const std::string CYAN    = "\033[36m";
    const std::string WHITE   = "\033[37m";
}

// ============================================================================
// Core Student Model used across multiple modules to maintain record consistency
// ============================================================================
struct Student {
    int id;                     // Unique student integer ID
    std::string name;           // Student's full name
    std::string division;       // Assigned division (e.g., "A", "B")
    std::string contact;        // Contact number as string to preserve leading zeros
    std::vector<double> marks;  // Scores in registered subjects (Maths, Physics, Chem, CS)

    Student() : id(0), name(""), division(""), contact("") {
        marks = {0.0, 0.0, 0.0, 0.0};
    }

    Student(int sId, const std::string& sName, const std::string& sDiv,
            const std::string& sContact, const std::vector<double>& sMarks)
        : id(sId), name(sName), division(sDiv), contact(sContact), marks(sMarks) {}

    // Computes aggregate total marks
    double getTotalMarks() const {
        double sum = 0.0;
        for (double m : marks) sum += m;
        return sum;
    }

    // Computes percentage / average marks
    double getAverageMarks() const {
        if (marks.empty()) return 0.0;
        return getTotalMarks() / marks.size();
    }

    // Formatted row representation for tables
    void printRow() const {
        std::cout << "| " << std::setw(6) << id
                  << " | " << std::setw(18) << std::left << name.substr(0, 18) << std::right
                  << " | " << std::setw(4) << division
                  << " | " << std::setw(12) << contact
                  << " | " << std::fixed << std::setprecision(1) << std::setw(6) << getAverageMarks() << "%"
                  << " | ";
        for (size_t i = 0; i < marks.size(); ++i) {
            std::cout << std::setw(4) << (int)marks[i] << (i + 1 < marks.size() ? "," : "");
        }
        std::cout << " |\n";
    }
};

// ============================================================================
// Safe Input Helper Functions (Defends against infinite loops on invalid input)
// ============================================================================
namespace InputHelper {
    inline void clearBuffer() {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    inline int getInteger(const std::string& prompt) {
        int value;
        while (true) {
            std::cout << prompt;
            if (std::cin >> value) {
                clearBuffer();
                return value;
            }
            std::cout << Color::RED << " [!] Invalid input. Please enter a valid integer.\n" << Color::RESET;
            clearBuffer();
        }
    }

    inline double getDouble(const std::string& prompt) {
        double value;
        while (true) {
            std::cout << prompt;
            if (std::cin >> value) {
                clearBuffer();
                return value;
            }
            std::cout << Color::RED << " [!] Invalid input. Please enter a valid number.\n" << Color::RESET;
            clearBuffer();
        }
    }
```

```
    inline std::string getString(const std::string& prompt) {
        std::string line;
        std::cout << prompt;
        std::getline(std::cin, line);
        // Trim leading and trailing whitespaces
        size_t first = line.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        size_t last = line.find_last_not_of(" \t\r\n");
        return line.substr(first, (last - first + 1));
    }

    inline void pause() {
        std::cout << Color::YELLOW << "\nPress Enter to continue..." << Color::RESET;
        std::string dummy;
        std::getline(std::cin, dummy);
    }
}

#endif // COMMON_HPP
```

```cpp
#ifndef MODULE1_STUDENT_SEARCH_HPP
#define MODULE1_STUDENT_SEARCH_HPP

#include "Common.hpp"
#include <algorithm>
#include <cctype>

class StudentManager {
private:
    std::vector<Student> students;

    // Helper: Case-insensitive string search
    static bool containsIgnoreCase(const std::string& str, const std::string& query) {
        if (query.empty()) return true;
        auto it = std::search(
            str.begin(), str.end(),
            query.begin(), query.end(),
            [](char ch1, char ch2) { return std::toupper(ch1) == std::toupper(ch2); }
        );
        return (it != str.end());
    }

public:
    StudentManager() {}

    // Direct access for integration with other modules
    std::vector<Student>& getStudents() { return students; }
    const std::vector<Student>& getStudents() const { return students; }

    // Check if a student ID already exists
    bool exists(int id) const {
        for (const auto& s : students) {
            if (s.id == id) return true;
        }
        return false;
    }

    // 1. Add Student Record with uniqueness check
    bool addStudent(const Student& s) {
        if (exists(s.id)) {
            std::cout << Color::RED << " [Error] Student ID " << s.id << " already exists! Duplicates are not allowed.\n" << Color::RESET;
            return false;
        }
        students.push_back(s);
        std::cout << Color::GREEN << " [Success] Student record for \"" << s.name << "\" (ID: " << s.id << ") added successfully.\n" << Color::RESET;
        return true;
    }

    // 2. Display all registered students in a formatted table
    void displayAll() const {
        if (students.empty()) {
            std::cout << Color::YELLOW << " [Notice] No student records found. The database is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "+------------+----------------------+------+----------------+---------+--------------------------+\n"
                  << "| ID         | Name                 | Div  | Contact        | Average | Marks (M, P, C, CS) |\n"
                  << "+------------+----------------------+------+----------------+---------+--------------------------+\n"
                  << Color::RESET;
        for (const auto& s : students) {
            s.printRow();
        }
        std::cout << Color::CYAN
                  << "+------------+----------------------+------+----------------+---------+--------------------------+\n"
                  << " Total Students Registered: " << students.size() << Color::RESET << "\n";
    }

    // 3. Linear Search by Student ID
    // Time Complexity: Best: O(1) [First item], Worst/Average: O(N) [Last item or not found]
    int linearSearchById(int targetId, int& comparisons) const {
        comparisons = 0;
        for (size_t i = 0; i < students.size(); ++i) {
            comparisons++;
            if (students[i].id == targetId) {
                return static_cast<int>(i);
            }
        }
        return -1; // Not found
    }

    // 4. Binary Search by Student ID
    // Requirement: Array MUST be sorted by ID first.
    // Time Complexity: Best: O(1) [Middle item], Worst/Average: O(log N)
    int binarySearchById(int targetId, int& comparisons) {
        comparisons = 0;
        if (students.empty()) return -1;

        // Ensure records are sorted by ID before binary search
        std::sort(students.begin(), students.end(), [](const Student& a, const Student& b) {
            return a.id < b.id;
        });

        int low = 0;
        int high = static_cast<int>(students.size()) - 1;

        while (low <= high) {
            comparisons++;
            int mid = low + (high - low) / 2;

            if (students[mid].id == targetId) {
                return mid; // Found
            } else if (students[mid].id < targetId) {
                low = mid + 1; // Discard left half
            } else {
                high = mid - 1; // Discard right half
            }
```

```
        }
        return -1; // Target not present
    }

    // 5. Search by Name (supports partial string matching)
    void searchByName(const std::string& nameQuery) const {
        if (students.empty()) {
            std::cout << Color::YELLOW << " [Notice] Student list is empty.\n" << Color::RESET;
            return;
        }

        std::vector<Student> matches;
        for (const auto& s : students) {
            if (containsIgnoreCase(s.name, nameQuery)) {
                matches.push_back(s);
            }
        }

        if (matches.empty()) {
            std::cout << Color::RED << " [Result] No student found matching \"" << nameQuery << "\".\n" << Color::RESET;
        } else {
            std::cout << Color::GREEN << " [Result] Found " << matches.size() << " match(es) for \"" << nameQuery << "\":\n" << Color::RESET;
            std::cout << Color::CYAN
                      << "+--------+----------------------+-------+----------------+---------+-----------------------+\n"
                      << "| ID     | Name                 | Div   | Contact        | Average | Marks (M, P, C, CS) |\n"
                      << "+--------+----------------------+-------+----------------+---------+-----------------------+\n"
                      << Color::RESET;
            for (const auto& s : matches) {
                s.printRow();
            }
            std::cout << Color::CYAN << "+--------+----------------------+-------+----------------+---------+-----------------------+\n" << Color::RESET;
        }
    }

    // 6. Update an existing student's details
    bool updateStudent(int id) {
        int dummyComps = 0;
        int index = linearSearchById(id, dummyComps);
        if (index == -1) {
            std::cout << Color::RED << " [Error] Student with ID " << id << " not found in the database.\n" << Color::RESET;
            return false;
        }

        Student& s = students[index];
        std::cout << Color::CYAN << " [Editing Student: " << s.name << " (ID: " << s.id << ")]\n" << Color::RESET;
        std::cout << " 1. Update Name\n"
                  << " 2. Update Division\n"
                  << " 3. Update Contact Number\n"
                  << " 4. Update Subject Marks\n"
                  << " 5. Cancel\n";
        int choice = InputHelper::getInteger(" Select field to update (1-5): ");

        switch (choice) {
            case 1: {
                std::string newName = InputHelper::getString(" Enter new Name: ");
                if (!newName.empty()) {
                    s.name = newName;
                    std::cout << Color::GREEN << " [Updated] Student name updated.\n" << Color::RESET;
                }
                break;
            }
            case 2: {
                std::string newDiv = InputHelper::getString(" Enter new Division: ");
                if (!newDiv.empty()) {
                    s.division = newDiv;
                    std::cout << Color::GREEN << " [Updated] Division updated.\n" << Color::RESET;
                }
                break;
            }
            case 3: {
                std::string newContact = InputHelper::getString(" Enter new Contact Number: ");
                if (!newContact.empty()) {
                    s.contact = newContact;
                    std::cout << Color::GREEN << " [Updated] Contact updated.\n" << Color::RESET;
                }
                break;
            }
            case 4: {
                std::cout << " Enter marks for 4 subjects (Maths, Physics, Chemistry, Computer Science):\n";
                for (int i = 0; i < 4; ++i) {
                    double mark = InputHelper::getDouble("  Subject " + std::to_string(i + 1) + " (0-100): ");
                    if (mark < 0) mark = 0;
                    if (mark > 100) mark = 100;
                    s.marks[i] = mark;
                }
                std::cout << Color::GREEN << " [Updated] Subject marks updated.\n" << Color::RESET;
                break;
            }
            default:
                std::cout << Color::YELLOW << " [Info] Update cancelled.\n" << Color::RESET;
                return false;
        }
        return true;
    }

    // 7. Delete a student record by ID (returns deleted student for undo stack tracking)
    bool deleteStudent(int id, Student& deletedRecord) {
        for (auto it = students.begin(); it != students.end(); ++it) {
            if (it->id == id) {
                deletedRecord = *it;
                students.erase(it);
                std::cout << Color::GREEN << " [Success] Student ID " << id << " deleted successfully.\n" << Color::RESET;
                return true;
            }
        }
        std::cout << Color::RED << " [Error] Student with ID " << id << " does not exist.\n" << Color::RESET;
        return false;
    }

    // Interactive Menu Handler for Module 1
    void runMenu() {
        while (true) {
```

```
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "═══════════════════════════════════════════════════════\n"
                      << "  MODULE 1: STUDENT INFORMATION MANAGEMENT & SEARCHING  \n"
                      << "═══════════════════════════════════════════════════════\n"
                      << Color::RESET;
            std::cout << " 1. Add New Student Record\n"
                      << " 2. Display All Student Records\n"
                      << " 3. Search Student by ID (Linear Search)\n"
                      << " 4. Search Student by ID (Binary Search - Requires Sorted Array)\n"
                      << " 5. Search Student by Name\n"
                      << " 6. Update Student Details\n"
                      << " 7. Delete Student Record\n"
                      << " 8. Algorithmic Complexity Comparison (Linear vs Binary Search)\n"
                      << " 0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-8): ");

            if (choice == 0) break;

            switch (choice) {
                case 1: {
                    std::cout << "\n" << Color::CYAN << "--- Add New Student Record ---" << Color::RESET << "\n";
                    int id = InputHelper::getInteger(" Enter Student ID (Unique Integer): ");
                    if (exists(id)) {
                        std::cout << Color::RED << " [Error] Student ID " << id << " already exists!\n" << Color::RESET;
                        break;
                    }
                    std::string name = InputHelper::getString(" Enter Student Full Name: ");
                    std::string div = InputHelper::getString(" Enter Division: ");
                    std::string contact = InputHelper::getString(" Enter Contact Number: ");
                    std::vector<double> marks(4);
                    std::cout << " Enter Marks for 4 Subjects (0-100):\n";
                    std::string subNames[4] = {"Mathematics", "Physics", "Chemistry", "Computer Science"};
                    for (int i = 0; i < 4; ++i) {
                        marks[i] = InputHelper::getDouble("  " + subNames[i] + ": ");
                        if (marks[i] < 0) marks[i] = 0;
                        if (marks[i] > 100) marks[i] = 100;
                    }
                    addStudent(Student(id, name, div, contact, marks));
                    break;
                }
                case 2:
                    displayAll();
                    break;
                case 3: {
                    if (students.empty()) {
                        std::cout << Color::YELLOW << " [Notice] Database is empty.\n" << Color::RESET;
                        break;
                    }
                    int targetId = InputHelper::getInteger(" Enter Student ID to search (Linear Search): ");
                    int comps = 0;
                    int idx = linearSearchById(targetId, comps);
                    if (idx != -1) {
                        std::cout << Color::GREEN << " [Found] Student details located at index " << idx << ":\n" << Color::RESET;
                        std::cout << Color::CYAN
                                  << "+--------+----------------------+------+----------------+---------+----------------------+\n"
                                  << "| ID     | Name                 | Div  | Contact        | Average | Marks (M, P, C, CS) |\n"
                                  << "+--------+----------------------+------+----------------+---------+----------------------+\n"
                                  << Color::RESET;
                        students[idx].printRow();
                        std::cout << Color::CYAN << "+--------+----------------------+------+----------------+---------+----------------------+\n" << Color::RESET;
                    } else {
                        std::cout << Color::RED << " [Not Found] Student ID " << targetId << " not present.\n" << Color::RESET;
                    }
                    std::cout << Color::MAGENTA << " --> Linear Search Comparisons made: " << comps << " (O(N) search)\n" << Color::RESET;
                    break;
                }
                case 4: {
                    if (students.empty()) {
                        std::cout << Color::YELLOW << " [Notice] Database is empty.\n" << Color::RESET;
                        break;
                    }
                    int targetId = InputHelper::getInteger(" Enter Student ID to search (Binary Search): ");
                    int comps = 0;
                    int idx = binarySearchById(targetId, comps);
                    if (idx != -1) {
                        std::cout << Color::GREEN << " [Found] Record located via Binary Search (after ID sort) at index " << idx << ":\n" << Color::RESET;
                        std::cout << Color::CYAN
                                  << "+--------+----------------------+------+----------------+---------+----------------------+\n"
                                  << "| ID     | Name                 | Div  | Contact        | Average | Marks (M, P, C, CS) |\n"
                                  << "+--------+----------------------+------+----------------+---------+----------------------+\n"
                                  << Color::RESET;
                        students[idx].printRow();
                        std::cout << Color::CYAN << "+--------+----------------------+------+----------------+---------+----------------------+\n" << Color::RESET;
                    } else {
                        std::cout << Color::RED << " [Not Found] Student ID " << targetId << " not present in database.\n" << Color::RESET;
                    }
                    std::cout << Color::MAGENTA << " --> Binary Search Comparisons made: " << comps << " (O(log N) search on sorted data)\n" << Color::RESET;
                    break;
                }
                case 5: {
                    std::string q = InputHelper::getString(" Enter Student Name (or substring) to search: ");
                    searchByName(q);
                    break;
                }
                case 6: {
                    int id = InputHelper::getInteger(" Enter Student ID to update: ");
                    updateStudent(id);
                    break;
                }
                case 7: {
                    int id = InputHelper::getInteger(" Enter Student ID to delete: ");
                    Student dummy;
                    deleteStudent(id, dummy);
                    break;
                }
                case 8: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "═══════════════════════════════════════════════════════════════════════\n"
                              << "         THEORETICAL ANALYSIS: LINEAR SEARCH vs BINARY SEARCH           \n"
                              << "═══════════════════════════════════════════════════════════════════════\n"
                              << Color::RESET
```

```
                              << " 1. LINEAR SEARCH:\n"
                              << "    - Best Case Time:    O(1) (Key found at first index)\n"
                              << "    - Average Case Time: O(N) (Key found in middle, N/2 comparisons)\n"
                              << "    - Worst Case Time:   O(N) (Key at last index or absent)\n"
                              << "    - Auxiliary Space:   O(1) (Iterative)\n"
                              << "    - Requirement:       None. Works on unsorted, random datasets.\n\n"
                              << " 2. BINARY SEARCH:\n"
                              << "    - Best Case Time:    O(1) (Key located directly at mid)\n"
                              << "    - Average Case Time: O(log2 N)\n"
                              << "    - Worst Case Time:   O(log2 N)\n"
                              << "    - Auxiliary Space:   O(1) (Iterative)\n"
                              << "    - Requirement:       STRICT PRE-CONDITION: Data MUST be sorted.\n"
                              << "    - Why Sorted?:       Binary search relies on monotonicity to discard\n"
                              << "                         half the search space in every comparison. If\n"
                              << "                         data is unsorted, determining whether target is\n"
                              << "                         to the left or right of mid is impossible.\n"
                              << "═══════════════════════════════════════════════════════════════════════\n";
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 8.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE1_STUDENT_SEARCH_HPP
```

```cpp
#ifndef MODULE2_SORTING_FILES_HPP
#define MODULE2_SORTING_FILES_HPP

#include "Common.hpp"
#include <fstream>
#include <sstream>

class SortingAndFileManager {
public:
    // ============================================================================
    // 1. Bubble Sort: Sort student records by Aggregate Percentage/Marks
    // Stable sort algorithm. Compares adjacent elements and bubbles largest.
    // Optimized with 'swapped' flag for O(N) best case when already sorted.
    // ============================================================================
    static void bubbleSortByMarks(std::vector<Student>& arr, long long& comparisons, long long& swaps) {
        comparisons = 0;
        swaps = 0;
        size_t n = arr.size();
        if (n <= 1) return;

        for (size_t i = 0; i < n - 1; ++i) {
            bool swapped = false;
            for (size_t j = 0; j < n - i - 1; ++j) {
                comparisons++;
                // Descending order (highest marks first)
                if (arr[j].getAverageMarks() < arr[j + 1].getAverageMarks()) {
                    std::swap(arr[j], arr[j + 1]);
                    swaps++;
                    swapped = true;
                }
            }
            // If no elements were swapped, array is already sorted!
            if (!swapped) break;
        }
    }

    // ============================================================================
    // 2. Selection Sort: Sort student records by Student ID (Ascending)
    // Selects minimum element in remaining unsorted portion and swaps with current.
    // Comparisons: O(N^2) always. Swaps: O(N) maximum.
    // ============================================================================
    static void selectionSortById(std::vector<Student>& arr, long long& comparisons, long long& swaps) {
        comparisons = 0;
        swaps = 0;
        size_t n = arr.size();
        if (n <= 1) return;

        for (size_t i = 0; i < n - 1; ++i) {
            size_t minIdx = i;
            for (size_t j = i + 1; j < n; ++j) {
                comparisons++;
                if (arr[j].id < arr[minIdx].id) {
                    minIdx = j;
                }
            }
            if (minIdx != i) {
                std::swap(arr[i], arr[minIdx]);
                swaps++;
            }
        }
    }

    // ============================================================================
    // 3. Insertion Sort: Sort records Alphabetically by Name (Ascending)
    // Builds sorted array one element at a time by shifting larger elements.
    // Best Case: O(N) if already sorted. Worst Case: O(N^2) if reverse sorted.
    // ============================================================================
    static void insertionSortByName(std::vector<Student>& arr, long long& comparisons, long long& shifts) {
        comparisons = 0;
        shifts = 0;
        size_t n = arr.size();
        if (n <= 1) return;

        for (size_t i = 1; i < n; ++i) {
            Student key = arr[i];
            int j = static_cast<int>(i) - 1;

            // Shift elements of arr[0..i-1] that are greater than key.name
            while (j >= 0) {
                comparisons++;
                if (arr[j].name > key.name) {
                    arr[j + 1] = arr[j];
                    shifts++;
                    j--;
                } else {
                    break;
                }
            }
            arr[j + 1] = key;
        }
    }

    // ============================================================================
    // Insertion Sort for integer grades vector
    // ============================================================================
    static void insertionSortGrades(std::vector<int>& grades, long long& comparisons, long long& shifts) {
        comparisons = 0;
        shifts = 0;
        size_t n = grades.size();
        for (size_t i = 1; i < n; ++i) {
            int key = grades[i];
            int j = static_cast<int>(i) - 1;
            while (j >= 0) {
                comparisons++;
                if (grades[j] > key) {
                    grades[j + 1] = grades[j];
```

```
                    shifts++;
                    j--;
                } else {
                    break;
                }
            }
            grades[j + 1] = key;
        }
    }

    // ============================================================
    // 4. File Handling: Read grades.txt, sort using insertion sort, write to sorted_grades.txt
    // ============================================================
    static void processGradeFiles(const std::string& inputFilename = "grades.txt",
                                  const std::string& outputFilename = "sorted_grades.txt") {
        std::cout << "\n" << Color::CYAN << "--- Grade File Processing via Insertion Sort ---" << Color::RESET << "\n";
        std::ifstream inFile(inputFilename);
        if (!inFile.is_open()) {
            std::cout << Color::RED << " [Error] Could not open file \"" << inputFilename << "\" for reading.\n"
                      << " Generating a sample \"" << inputFilename << "\" with test data now...\n" << Color::RESET;
            // Create a sample file so the user doesn't get blocked
            std::ofstream sampleFile(inputFilename);
            sampleFile << "85\n42\n99\n76\n58\n91\n63\n88\n70\n100\n35\n";
            sampleFile.close();
            inFile.open(inputFilename);
            if (!inFile.is_open()) {
                std::cout << Color::RED << " [Fatal] Unable to create or read \"" << inputFilename << "\".\n" << Color::RESET;
                return;
            }
            std::cout << Color::GREEN << " [Success] Sample \"" << inputFilename << "\" created successfully.\n" << Color::RESET;
        }

        std::vector<int> grades;
        std::string line;
        int lineNum = 0;
        int invalidCount = 0;

        while (std::getline(inFile, line)) {
            lineNum++;
            // Trim whitespace
            size_t first = line.find_first_not_of(" \t\r\n");
            if (first == std::string::npos) continue; // Skip blank lines
            size_t last = line.find_last_not_of(" \t\r\n");
            std::string trimmed = line.substr(first, (last - first + 1));

            std::stringstream ss(trimmed);
            int gradeVal;
            if (ss >> gradeVal && ss.eof()) {
                if (gradeVal >= 0 && gradeVal <= 100) {
                    grades.push_back(gradeVal);
                } else {
                    std::cout << Color::YELLOW << " [Warning] Line " << lineNum << ": Grade " << gradeVal
                              << " out of bounds (0-100). Ignored.\n" << Color::RESET;
                    invalidCount++;
                }
            } else {
                std::cout << Color::YELLOW << " [Warning] Line " << lineNum << ": Invalid entry \"" << trimmed << "\" ignored.\n" << Color::RESET;
                invalidCount++;
            }
        }
        inFile.close();

        if (grades.empty()) {
            std::cout << Color::RED << " [Error] File \"" << inputFilename << "\" contained no valid integer grades.\n" << Color::RESET;
            return;
        }

        std::cout << Color::GREEN << " [Success] Read " << grades.size() << " valid grades from \""
                  << inputFilename << "\" (" << invalidCount << " invalid ignored).\n" << Color::RESET;

        std::cout << " Raw Unsorted Grades: ";
        for (int g : grades) std::cout << g << " ";
        std::cout << "\n";

        // Sort using manual Insertion Sort
        long long comps = 0, shifts = 0;
        insertionSortGrades(grades, comps, shifts);

        std::cout << Color::GREEN << " Sorted Grades (Ascending): ";
        for (int g : grades) std::cout << g << " ";
        std::cout << Color::RESET << "\n";
        std::cout << Color::MAGENTA << " -> Insertion Sort Metric: " << comps << " comparisons, " << shifts << " shifts.\n" << Color::RESET;

        // Write output
        std::ofstream outFile(outputFilename);
        if (!outFile.is_open()) {
            std::cout << Color::RED << " [Error] Could not open \"" << outputFilename << "\" for writing.\n" << Color::RESET;
            return;
        }

        for (int g : grades) {
            outFile << g << "\n";
        }
        outFile.close();
        std::cout << Color::GREEN << " [Success] Sorted grades written to \"" << outputFilename << "\" successfully.\n" << Color::RESET;
    }

    // Interactive Menu Handler for Module 2
    static void runMenu(std::vector<Student>& sharedStudents) {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "============================================================\n"
                      << "      MODULE 2: SORTING ALGORITHMS & FILE HANDLING      \n"
                      << "============================================================\n"
                      << Color::RESET;
            std::cout << " 1. Bubble Sort: Sort Students by Marks/Percentage (Descending)\n"
                      << " 2. Selection Sort: Sort Students by Student ID (Ascending)\n"
                      << " 3. Insertion Sort: Sort Students Alphabetically by Name (A-Z)\n"
                      << " 4. File Handling: Read 'grades.txt', Sort via Insertion Sort & Save\n"
                      << " 5. Display Current Student List Order\n"
                      << " 6. Algorithmic Complexity Comparison (Bubble vs Selection vs Insertion)\n"
                      << " 0. Return to Main Menu\n";
```

```
            int choice = InputHelper::getInteger(" Enter your choice (0-6): ");
            if (choice == 0) break;

            switch (choice) {
                case 1: {
                    if (sharedStudents.empty()) {
                        std::cout << Color::YELLOW << " [Notice] No student records available to sort.\n" << Color::RESET;
                        break;
                    }
                    long long comps = 0, swaps = 0;
                    bubbleSortByMarks(sharedStudents, comps, swaps);
                    std::cout << Color::GREEN << " [Sorted] Student records sorted by Average Marks (Bubble Sort):\n" << Color::RESET;
                    std::cout << Color::MAGENTA << " -> Comparisons: " << comps << " | Swaps: " << swaps << "\n" << Color::RESET;
                    displayTable(sharedStudents);
                    break;
                }
                case 2: {
                    if (sharedStudents.empty()) {
                        std::cout << Color::YELLOW << " [Notice] No student records available to sort.\n" << Color::RESET;
                        break;
                    }
                    long long comps = 0, swaps = 0;
                    selectionSortById(sharedStudents, comps, swaps);
                    std::cout << Color::GREEN << " [Sorted] Student records sorted by Student ID (Selection Sort):\n" << Color::RESET;
                    std::cout << Color::MAGENTA << " -> Comparisons: " << comps << " | Swaps: " << swaps << "\n" << Color::RESET;
                    displayTable(sharedStudents);
                    break;
                }
                case 3: {
                    if (sharedStudents.empty()) {
                        std::cout << Color::YELLOW << " [Notice] No student records available to sort.\n" << Color::RESET;
                        break;
                    }
                    long long comps = 0, shifts = 0;
                    insertionSortByName(sharedStudents, comps, shifts);
                    std::cout << Color::GREEN << " [Sorted] Student records sorted Alphabetically by Name (Insertion Sort):\n" << Color::RESET;
                    std::cout << Color::MAGENTA << " -> Comparisons: " << comps << " | Shifts: " << shifts << "\n" << Color::RESET;
                    displayTable(sharedStudents);
                    break;
                }
                case 4:
                    processGradeFiles();
                    break;
                case 5:
                    displayTable(sharedStudents);
                    break;
                case 6: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "=============================================================================================\n"
                              << "                          SORTING ALGORITHMS COMPLEXITY COMPARISON                           \n"
                              << "=============================================================================================\n"
                              << Color::RESET
                              << " Algorithm      | Best Time | Avg Time  | Worst Time | Aux Space | Stability | Main Advantage\n"
                              << "-------------------+-----------+-----------+------------+-----------+-----------+-------------------------\n"
                              << " Bubble Sort    | O(N)      | O(N^2)    | O(N^2)     | O(1)      | Stable    | Detects already sorted data\n"
                              << " Selection Sort | O(N^2)    | O(N^2)    | O(N^2)     | O(1)      | Unstable  | Minimizes swaps (at most N)\n"
                              << " Insertion Sort | O(N)      | O(N^2)    | O(N^2)     | O(1)      | Stable    | Optimal for nearly sorted\n"
                              << "-------------------+-----------+-----------+------------+-----------+-----------+-------------------------\n"
                              << Color::CYAN
                              << " [!] EFFECT OF INPUT ON INSERTION SORT:\n"
                              << "   1. Already Sorted: Each element compared once with left predecessor, zero shifts.\n"
                              << "      Total Time = O(N) [Optimal linear benchmark].\n"
                              << "   2. Reverse Sorted: Every element must shift all predecessors to the right.\n"
                              << "      Total Comparisons & Shifts = N*(N-1)/2 = O(N^2) [Worst case scenario].\n"
                              << "=============================================================================================\n"
                              << Color::RESET;
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 6.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }

private:
    static void displayTable(const std::vector<Student>& students) {
        if (students.empty()) {
            std::cout << Color::YELLOW << " [Notice] Student list is empty.\n" << Color::RESET;
            return;
        }
        std::cout << Color::CYAN << Color::BOLD
                  << "+--------+------------------------+-------+----------------+---------+-------------------------+\n"
                  << "| ID     | Name                   | Div   | Contact        | Average | Marks (M, P, C, CS) |\n"
                  << "+--------+------------------------+-------+----------------+---------+-------------------------+\n"
                  << Color::RESET;
        for (const auto& s : students) {
            s.printRow();
        }
        std::cout << Color::CYAN << "+--------+------------------------+-------+----------------+---------+-------------------------+\n" << Color::RESET;
    }
};

#endif // MODULE2_SORTING_FILES_HPP
```

```cpp
#ifndef MODULE3_SPARSE_MATRIX_HPP
#define MODULE3_SPARSE_MATRIX_HPP

#include "Common.hpp"

// Single Triplet entry (Row, Column, Value)
struct MatrixTriplet {
    int row;
    int col;
    int val;
};

class SparseGradeMatrix {
private:
    int numStudents;
    int numSubjects;
    std::vector<std::string> studentLabels;
    std::vector<std::string> subjectLabels;
    std::vector<std::vector<int>> matrix;   // 2D Array Representation

    // Triplet Representation:
    // triplets[0] = {numStudents, numSubjects, totalNonZeroElements}
    // triplets[1..NZ] = {rowIndex, colIndex, mark}
    std::vector<MatrixTriplet> triplets;

public:
    SparseGradeMatrix() {
        loadDefaultSample();
    }

    void loadDefaultSample() {
        numStudents = 5;
        numSubjects = 4;
        studentLabels = {"S1 (Aarav)", "S2 (Diya)", "S3 (Rohan)", "S4 (Sneha)", "S5 (Ishaan)"};
        subjectLabels = {"Mathematics", "Physics", "Chemistry", "Computer Sci"};

        matrix = {
            {85,  0, 78, 90},
            { 0, 76, 82,  0},
            {91, 88,  0, 95},
            { 0,  0, 74, 80},
            {79, 84,  0,  0}
        };

        convertToTriplet();
    }

    // Convert standard 2D matrix into compact 3-Tuple (Triplet) format
    void convertToTriplet() {
        triplets.clear();
        int nzCount = 0;

        // First pass: count non-zeros and populate
        // Reserve index 0 for matrix header metadata
        triplets.push_back({numStudents, numSubjects, 0});

        for (int r = 0; r < numStudents; ++r) {
            for (int c = 0; c < numSubjects; ++c) {
                if (matrix[r][c] != 0) {
                    triplets.push_back({r, c, matrix[r][c]});
                    nzCount++;
                }
            }
        }
        triplets[0].val = nzCount; // Store total non-zero count at header
    }

    // Display original 2D dense matrix
    void displayDenseMatrix() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Dense 2D Grade Matrix (0 = Not Enrolled) ---" << Color::RESET << "\n";
        std::cout << std::left << std::setw(16) << "Student";
        for (const auto& sub : subjectLabels) {
            std::cout << std::right << std::setw(14) << sub;
        }
        std::cout << "\n" << std::string(16 + numSubjects * 14, '-') << "\n";

        for (int r = 0; r < numStudents; ++r) {
            std::cout << std::left << std::setw(16) << studentLabels[r];
            for (int c = 0; c < numSubjects; ++c) {
                if (matrix[r][c] == 0) {
                    std::cout << std::right << std::setw(14) << "-";
                } else {
                    std::cout << std::right << std::setw(14) << matrix[r][c];
                }
            }
            std::cout << "\n";
        }
        std::cout << std::string(16 + numSubjects * 14, '-') << "\n";
    }

    // Display Array-based Triplet Representation
    void displayTripletRepresentation() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Triplet (3-Tuple) Sparse Representation ---" << Color::RESET << "\n";
        std::cout << "+-------+-------+-------+-----------------------------------+\n"
                  << "| Row   | Col   | Value | Description                       |\n"
                  << "+-------+-------+-------+-----------------------------------+\n";

        // Header triplet
        std::cout << "| " << std::setw(5) << triplets[0].row
                  << " | " << std::setw(5) << triplets[0].col
                  << " | " << std::setw(5) << triplets[0].val
                  << " | " << std::setw(31) << std::left << "Header: [Rows, Cols, NonZeros]" << std::right << " |\n";
        std::cout << "+-------+-------+-------+-----------------------------------+\n";

        for (size_t i = 1; i < triplets.size(); ++i) {
            std::string desc = studentLabels[triplets[i].row] + " → " + subjectLabels[triplets[i].col];
```

```
            std::cout << "| " << std::setw(5) << triplets[i].row
                      << " | " << std::setw(5) << triplets[i].col
                      << " | " << std::setw(5) << triplets[i].val
                      << " | " << std::setw(31) << std::left << desc.substr(0, 31) << std::right << " |\n";
        }
        std::cout << "+-------+-------+-------+---------------------------------+\n";
        std::cout << " Total elements: " << (numStudents * numSubjects)
                  << " | Non-Zero Entries: " << triplets[0].val
                  << " | Sparsity: " << std::fixed << std::setprecision(1)
                  << (100.0 * (1.0 - (double)triplets[0].val / (numStudents * numSubjects))) << "%\n";
    }

    // Reconstruct dense matrix from Triplet representation
    void reconstructFromTriplet() const {
        std::cout << "\n" << Color::GREEN << Color::BOLD
                  << "--- Reconstructed Matrix from Triplet Representation ---" << Color::RESET << "\n";

        int rows = triplets[0].row;
        int cols = triplets[0].col;
        int nonZeros = triplets[0].val;

        // Initialize empty matrix filled with 0s
        std::vector<std::vector<int>> recon(rows, std::vector<int>(cols, 0));

        // Populate values directly from triplet entries
        for (int i = 1; i <= nonZeros; ++i) {
            recon[triplets[i].row][triplets[i].col] = triplets[i].val;
        }

        // Print reconstructed matrix
        std::cout << std::left << std::setw(16) << "Student";
        for (const auto& sub : subjectLabels) {
            std::cout << std::right << std::setw(14) << sub;
        }
        std::cout << "\n" << std::string(16 + cols * 14, '-') << "\n";

        for (int r = 0; r < rows; ++r) {
            std::cout << std::left << std::setw(16) << studentLabels[r];
            for (int c = 0; c < cols; ++c) {
                if (recon[r][c] == 0) {
                    std::cout << std::right << std::setw(14) << "-";
                } else {
                    std::cout << std::right << std::setw(14) << recon[r][c];
                }
            }
            std::cout << "\n";
        }
        std::cout << std::string(16 + cols * 14, '-') << "\n";
        std::cout << Color::GREEN << " [Validation] Reconstructed matrix matches original dense matrix perfectly!\n" << Color::RESET;
    }

    // Subject-wise Averages (excluding 0 / un-enrolled students)
    void computeSubjectAverages() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Subject-Wise Academic Analysis ---" << Color::RESET << "\n";
        std::cout << "+------------------+------------+--------------+----------------+\n"
                  << "| Subject Name     | Enrolled   | Total Marks | Average Grade |\n"
                  << "+------------------+------------+--------------+----------------+\n";

        double highestAvg = -1.0;
        int highestAvgSubjectIdx = -1;

        for (int c = 0; c < numSubjects; ++c) {
            int enrolledCount = 0;
            int totalMarks = 0;
            for (int r = 0; r < numStudents; ++r) {
                if (matrix[r][c] > 0) {
                    enrolledCount++;
                    totalMarks += matrix[r][c];
                }
            }
            double avg = (enrolledCount > 0) ? ((double)totalMarks / enrolledCount) : 0.0;
            if (avg > highestAvg) {
                highestAvg = avg;
                highestAvgSubjectIdx = c;
            }

            std::cout << "| " << std::setw(16) << std::left << subjectLabels[c] << std::right
                      << " | " << std::setw(10) << enrolledCount
                      << " | " << std::setw(11) << totalMarks
                      << " | " << std::fixed << std::setprecision(2) << std::setw(13) << avg << " |\n";
        }
        std::cout << "+------------------+------------+--------------+----------------+\n";

        if (highestAvgSubjectIdx != -1) {
            std::cout << Color::GREEN << " [Highest Average Subject]: "
                      << subjectLabels[highestAvgSubjectIdx]
                      << " with an average score of " << std::fixed << std::setprecision(2) << highestAvg << "\n" << Color::RESET;
        }
    }

    // Identify student(s) with highest individual grade
    void findHighestIndividualGrade() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Top Performing Student(s) (Highest Individual Grade) ---" << Color::RESET << "\n";
        int maxGrade = -1;

        // Find global maximum
        for (int r = 0; r < numStudents; ++r) {
            for (int c = 0; c < numSubjects; ++c) {
                if (matrix[r][c] > maxGrade) {
                    maxGrade = matrix[r][c];
                }
            }
        }

        std::cout << " Highest Score Achieved on Campus: " << Color::GREEN << Color::BOLD << maxGrade << Color::RESET << "\n";
        std::cout << " Achieving Students & Subject(s):\n";

        for (int r = 0; r < numStudents; ++r) {
            for (int c = 0; c < numSubjects; ++c) {
                if (matrix[r][c] == maxGrade) {
                    std::cout << "  * " << studentLabels[r] << " in " << subjectLabels[c] << " (" << maxGrade << "/100)\n";
                }
```

```
            }
        }
    }

    // Storage Comparison and Sparsity Analysis
    void analyzeStorageEfficiency() const {
        int denseElements = numStudents * numSubjects;
        int denseBytes = denseElements * sizeof(int);

        int nonZeros = triplets[0].val;
        int tripletEntries = nonZeros + 1; // including header
        int tripletBytes = tripletEntries * 3 * sizeof(int);

        std::cout << "\n" << Color::YELLOW << Color::BOLD
                  << "═══════════════════════════════════════════════════════════════════════\n"
                  << "        STORAGE COMPLEXITY ANALYSIS: DENSE MATRIX vs TRIPLET            \n"
                  << "═══════════════════════════════════════════════════════════════════════\n"
                  << Color::RESET
                  << " 1. Dense 2D Array Representation:\n"
                  << "    - Total Cells:       " << numStudents << " rows x " << numSubjects << " cols = " << denseElements << " elements\n"
                  << "    - Memory Consumed:   " << denseElements << " x " << sizeof(int) << " bytes = " << denseBytes << " bytes\n\n"
                  << " 2. Triplet (3-Tuple) Representation:\n"
                  << "    - Non-Zero Entries:  " << nonZeros << " (plus 1 header tuple)\n"
                  << "    - Triplet Entries:   " << tripletEntries << " tuples x 3 integers\n"
                  << "    - Memory Consumed:   " << tripletEntries * 3 << " x " << sizeof(int) << " bytes = " << tripletBytes << " bytes\n\n"
                  << " 3. Memory Comparison for Current Dataset:\n"
                  << "    - Difference:        " << (denseBytes - tripletBytes) << " bytes ("
                  << (tripletBytes <= denseBytes ? "Triplet saves memory" : "Dense is more compact due to small N") << ")\n\n"
                  << Color::CYAN
                  << " [!] THEORETICAL BREAK-EVEN THRESHOLD:\n"
                  << "   Triplet stores 3 integers per non-zero element (Row, Col, Val).\n"
                  << "   Condition to save memory: 3 * (NZ + 1) < (Rows * Cols)\n"
                  << "   Equivalently: NZ < (Rows * Cols) / 3\n"
                  << "   Therefore, Sparse Triplet representation saves memory whenever the\n"
                  << "   matrix has LESS THAN ~33.3% non-zero elements (i.e., >66.7% sparse).\n"
                  << "   For large university grade sheets (e.g. 5,000 students x 60 electives),\n"
                  << "   students only enroll in 5-6 courses, meaning 90% sparsity, where\n"
                  << "   Triplet sparse storage saves over 70% of RAM!\n"
                  << "═══════════════════════════════════════════════════════════════════════\n"
                  << Color::RESET;
    }

    // Interactive Menu Handler for Module 3
    void runMenu() {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "═══════════════════════════════════════════════════\n"
                      << "   MODULE 3: SPARSE MATRIX REPRESENTATION & ANALYSIS    \n"
                      << "═══════════════════════════════════════════════════\n"
                      << Color::RESET;
            std::cout << " 1. Display Dense 2D Grade Matrix\n"
                      << " 2. Display 3-Tuple (Triplet) Sparse Matrix Representation\n"
                      << " 3. Reconstruct Dense Matrix from Triplet Format\n"
                      << " 4. Compute Subject-wise Averages (Excluding Unregistered)\n"
                      << " 5. Find Student(s) with Highest Individual Grade & Top Subject\n"
                      << " 6. Storage & Sparsity Complexity Analysis\n"
                      << " 7. Input Custom Sparse Matrix Values\n"
                      << " 0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-7): ");
            if (choice == 0) break;

            switch (choice) {
                case 1:
                    displayDenseMatrix();
                    break;
                case 2:
                    displayTripletRepresentation();
                    break;
                case 3:
                    reconstructFromTriplet();
                    break;
                case 4:
                    computeSubjectAverages();
                    break;
                case 5:
                    findHighestIndividualGrade();
                    break;
                case 6:
                    analyzeStorageEfficiency();
                    break;
                case 7: {
                    std::cout << "\n" << Color::CYAN << "--- Custom Matrix Entry ---" << Color::RESET << "\n";
                    numStudents = InputHelper::getInteger(" Enter number of students (rows 1-20): ");
                    if (numStudents < 1) numStudents = 1;
                    if (numStudents > 20) numStudents = 20;

                    numSubjects = InputHelper::getInteger(" Enter number of subjects (cols 1-10): ");
                    if (numSubjects < 1) numSubjects = 1;
                    if (numSubjects > 10) numSubjects = 10;

                    studentLabels.clear();
                    for (int i = 0; i < numStudents; ++i) {
                        studentLabels.push_back("S" + std::to_string(i + 1));
                    }
                    subjectLabels.clear();
                    for (int j = 0; j < numSubjects; ++j) {
                        subjectLabels.push_back("Sub" + std::to_string(j + 1));
                    }

                    matrix.assign(numStudents, std::vector<int>(numSubjects, 0));
                    std::cout << " Enter grades for each cell (enter 0 if student is not enrolled):\n";
                    for (int r = 0; r < numStudents; ++r) {
                        for (int c = 0; c < numSubjects; ++c) {
                            int g = InputHelper::getInteger("  " + studentLabels[r] + " in " + subjectLabels[c] + ": ");
                            if (g < 0) g = 0;
                            if (g > 100) g = 100;
                            matrix[r][c] = g;
                        }
                    }
                    convertToTriplet();
                    std::cout << Color::GREEN << " [Success] Custom matrix loaded and converted to Triplet format.\n" << Color::RESET;
```

```
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 7.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE3_SPARSE_MATRIX_HPP
```

```cpp
#ifndef MODULE4_LINKED_LISTS_HPP
#define MODULE4_LINKED_LISTS_HPP

#include "Common.hpp"

// ============================================================================
// PART A: SINGLY LINKED LIST (Campus Maintenance Requests)
// ============================================================================
struct MaintenanceRequest {
    int requestId;
    std::string requesterName;
    std::string location;
    std::string description;
    std::string status; // "Pending", "In Progress", "Completed"

    MaintenanceRequest(int id = 0, const std::string& name = "", const std::string& loc = "",
                       const std::string& desc = "", const std::string& stat = "Pending")
        : requestId(id), requesterName(name), location(loc), description(desc), status(stat) {}
};

struct SLLNode {
    MaintenanceRequest data;
    SLLNode* next;
    SLLNode(const MaintenanceRequest& req) : data(req), next(nullptr) {}
};

class SinglyLinkedList {
private:
    SLLNode* head;

public:
    SinglyLinkedList() : head(nullptr) {}
    ~SinglyLinkedList() { clear(); }

    void clear() {
        SLLNode* curr = head;
        while (curr) {
            SLLNode* temp = curr;
            curr = curr->next;
            delete temp;
        }
        head = nullptr;
    }

    bool exists(int id) const {
        SLLNode* curr = head;
        while (curr) {
            if (curr->data.requestId == id) return true;
            curr = curr->next;
        }
        return false;
    }

    // Add request at the end of the list
    void addRequest(const MaintenanceRequest& req) {
        if (exists(req.requestId)) {
            std::cout << Color::RED << " [Error] Request ID " << req.requestId << " already exists!\n" << Color::RESET;
            return;
        }
        SLLNode* newNode = new SLLNode(req);
        if (!head) {
            head = newNode;
        } else {
            SLLNode* curr = head;
            while (curr->next) {
                curr = curr->next;
            }
            curr->next = newNode;
        }
        std::cout << Color::GREEN << " [Success] Maintenance request #" << req.requestId << " logged.\n" << Color::RESET;
    }

    // Delete request by ID
    bool deleteRequest(int id) {
        if (!head) {
            std::cout << Color::YELLOW << " [Notice] Request list is empty. Nothing to delete.\n" << Color::RESET;
            return false;
        }

        // Case 1: Head node deletion
        if (head->data.requestId == id) {
            SLLNode* temp = head;
            head = head->next;
            delete temp;
            std::cout << Color::GREEN << " [Success] Request #" << id << " deleted.\n" << Color::RESET;
            return true;
        }

        // Case 2: Intermediate or tail node deletion
        SLLNode* curr = head;
        while (curr->next && curr->next->data.requestId != id) {
            curr = curr->next;
        }

        if (curr->next) {
            SLLNode* temp = curr->next;
            curr->next = curr->next->next;
            delete temp;
            std::cout << Color::GREEN << " [Success] Request #" << id << " deleted.\n" << Color::RESET;
            return true;
        }

        std::cout << Color::RED << " [Error] Request ID " << id << " not found.\n" << Color::RESET;
        return false;
    }

```

```
    // Search for a request by ID
    SLLNode* searchRequest(int id) const {
        SLLNode* curr = head;
        while (curr) {
            if (curr->data.requestId == id) return curr;
            curr = curr->next;
        }
        return nullptr;
    }

    // Update status of an existing request
    bool updateStatus(int id, const std::string& newStatus) {
        SLLNode* node = searchRequest(id);
        if (!node) {
            std::cout << Color::RED << " [Error] Request ID " << id << " not found.\n" << Color::RESET;
            return false;
        }
        node->data.status = newStatus;
        std::cout << Color::GREEN << " [Success] Request #" << id << " status updated to \"" << newStatus << "\".\n" << Color::RESET;
        return true;
    }

    // Display all requests in formatted table
    void displayAll() const {
        if (!head) {
            std::cout << Color::YELLOW << " [Notice] No maintenance requests logged.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "+------+--------------------+--------------------+--------------------------+--------------+\n"
                  << "| ID   | Requester          | Location           | Description              | Status       |\n"
                  << "+------+--------------------+--------------------+--------------------------+--------------+\n"
                  << Color::RESET;
        SLLNode* curr = head;
        while (curr) {
            std::cout << "| " << std::setw(3) << curr->data.requestId
                      << " | " << std::setw(18) << std::left << curr->data.requesterName.substr(0, 18) << std::right
                      << " | " << std::setw(18) << std::left << curr->data.location.substr(0, 18) << std::right
                      << " | " << std::setw(25) << std::left << curr->data.description.substr(0, 25) << std::right
                      << " | " << std::setw(12) << std::left << curr->data.status << std::right
                      << " |\n";
            curr = curr->next;
        }
        std::cout << Color::CYAN << "+------+--------------------+--------------------+--------------------------+--------------+\n" << Color::RESET;
    }

    // Count pending requests
    int countPending() const {
        int count = 0;
        SLLNode* curr = head;
        while (curr) {
            if (curr->data.status == "Pending") {
                count++;
            }
            curr = curr->next;
        }
        return count;
    }
};

// ============================================================================
// PART B: DOUBLY LINKED LIST (Campus Shuttle Route)
// ============================================================================
struct ShuttleStation {
    int stationId;
    std::string stationName;

    ShuttleStation(int id = 0, const std::string& name = "") : stationId(id), stationName(name) {}
};

struct DLLNode {
    ShuttleStation data;
    DLLNode* prev;
    DLLNode* next;
    DLLNode(const ShuttleStation& st) : data(st), prev(nullptr), next(nullptr) {}
};

class DoublyLinkedList {
private:
    DLLNode* head;
    DLLNode* tail;

public:
    DoublyLinkedList() : head(nullptr), tail(nullptr) {}
    ~DoublyLinkedList() { clear(); }

    void clear() {
        DLLNode* curr = head;
        while (curr) {
            DLLNode* temp = curr;
            curr = curr->next;
            delete temp;
        }
        head = tail = nullptr;
    }

    bool exists(int id) const {
        DLLNode* curr = head;
        while (curr) {
            if (curr->data.stationId == id) return true;
            curr = curr->next;
        }
        return false;
    }

    // 1. Insert at Beginning
    void insertAtBeginning(const ShuttleStation& st) {
        if (exists(st.stationId)) {
            std::cout << Color::RED << " [Error] Station ID " << st.stationId << " already exists!\n" << Color::RESET;
            return;
        }
        DLLNode* newNode = new DLLNode(st);
```

```
        if (!head) {
            head = tail = newNode;
        } else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
        std::cout << Color::GREEN << " [Success] Station \"" << st.stationName << "\" inserted at beginning.\n" << Color::RESET;
    }

    // 2. Insert at End
    void insertAtEnd(const ShuttleStation& st) {
        if (exists(st.stationId)) {
            std::cout << Color::RED << " [Error] Station ID " << st.stationId << " already exists!\n" << Color::RESET;
            return;
        }
        DLLNode* newNode = new DLLNode(st);
        if (!tail) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
        std::cout << Color::GREEN << " [Success] Station \"" << st.stationName << "\" inserted at end.\n" << Color::RESET;
    }

    // 3. Insert at Specified Position (1-based index)
    void insertAtPosition(const ShuttleStation& st, int pos) {
        if (pos <= 1) {
            insertAtBeginning(st);
            return;
        }
        if (exists(st.stationId)) {
            std::cout << Color::RED << " [Error] Station ID " << st.stationId << " already exists!\n" << Color::RESET;
            return;
        }

        DLLNode* curr = head;
        for (int i = 1; i < pos - 1 && curr; ++i) {
            curr = curr->next;
        }

        if (!curr || !curr->next) {
            // Append at end if position is beyond end
            insertAtEnd(st);
            return;
        }

        DLLNode* newNode = new DLLNode(st);
        newNode->next = curr->next;
        newNode->prev = curr;
        curr->next->prev = newNode;
        curr->next = newNode;
        std::cout << Color::GREEN << " [Success] Station \"" << st.stationName << "\" inserted at position " << pos << ".\n" << Color::RESET;
    }

    // 4. Delete Station by ID
    bool deleteStation(int id) {
        if (!head) {
            std::cout << Color::YELLOW << " [Notice] Route is empty.\n" << Color::RESET;
            return false;
        }

        DLLNode* curr = head;
        while (curr && curr->data.stationId != id) {
            curr = curr->next;
        }

        if (!curr) {
            std::cout << Color::RED << " [Error] Station ID " << id << " not found.\n" << Color::RESET;
            return false;
        }

        if (curr == head && curr == tail) {
            head = tail = nullptr;
        } else if (curr == head) {
            head = head->next;
            head->prev = nullptr;
        } else if (curr == tail) {
            tail = tail->prev;
            tail->next = nullptr;
        } else {
            curr->prev->next = curr->next;
            curr->next->prev = curr->prev;
        }

        delete curr;
        std::cout << Color::GREEN << " [Success] Station ID " << id << " removed from route.\n" << Color::RESET;
        return true;
    }

    // 5. Search for a Station
    DLLNode* searchStation(int id) const {
        DLLNode* curr = head;
        while (curr) {
            if (curr->data.stationId == id) return curr;
            curr = curr->next;
        }
        return nullptr;
    }

    // 6. Display in Forward Order (Head to Tail)
    void displayForward() const {
        if (!head) {
            std::cout << Color::YELLOW << " [Notice] Shuttle route is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD << " Forward Shuttle Route (Outbound):" << Color::RESET << "\n [START] ";
        DLLNode* curr = head;
        while (curr) {
            std::cout << "-> [" << curr->data.stationId << ": " << curr->data.stationName << "] ";
```

```
            curr = curr->next;
        }
        std::cout << "-> [TERMINUS]\n";
    }

    // 7. Display in Reverse Order (Tail to Head)
    void displayReverse() const {
        if (!tail) {
            std::cout << Color::YELLOW << " [Notice] Shuttle route is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::MAGENTA << Color::BOLD << " Reverse Shuttle Route (Inbound):" << Color::RESET << "\n [TERMINUS] ";
        DLLNode* curr = tail;
        while (curr) {
            std::cout << "-> [" << curr->data.stationId << ": " << curr->data.stationName << "] ";
            curr = curr->prev;
        }
        std::cout << "-> [START]\n";
    }
};

// ============================================================================
// PART C: SINGLY CIRCULAR LINKED LIST (Campus Announcement Playlist)
// ============================================================================
struct Announcement {
    int announcementId;
    std::string title;

    Announcement(int id = 0, const std::string& t = "") : announcementId(id), title(t) {}
};

struct CLLNode {
    Announcement data;
    CLLNode* next;
    CLLNode(const Announcement& ann) : data(ann), next(nullptr) {}
};

class CircularLinkedList {
private:
    CLLNode* last; // Tail pointer (last->next is head)

public:
    CircularLinkedList() : last(nullptr) {}
    ~CircularLinkedList() { clear(); }

    void clear() {
        if (!last) return;
        CLLNode* curr = last->next;
        last->next = nullptr; // Break the cycle to avoid infinite loop
        while (curr) {
            CLLNode* temp = curr;
            curr = curr->next;
            delete temp;
        }
        last = nullptr;
    }

    bool exists(int id) const {
        if (!last) return false;
        CLLNode* curr = last->next;
        do {
            if (curr->data.announcementId == id) return true;
            curr = curr->next;
        } while (curr != last->next);
        return false;
    }

    // Add Announcement to playlist
    void addAnnouncement(const Announcement& ann) {
        if (exists(ann.announcementId)) {
            std::cout << Color::RED << " [Error] Announcement ID " << ann.announcementId << " already exists!\n" << Color::RESET;
            return;
        }
        CLLNode* newNode = new CLLNode(ann);
        if (!last) {
            last = newNode;
            last->next = last; // Self-loop
        } else {
            newNode->next = last->next;
            last->next = newNode;
            last = newNode;
        }
        std::cout << Color::GREEN << " [Success] Announcement added to circular playlist: \"" << ann.title << "\"\n" << Color::RESET;
    }

    // Delete Announcement by ID
    bool deleteAnnouncement(int id) {
        if (!last) {
            std::cout << Color::YELLOW << " [Notice] Playlist is empty.\n" << Color::RESET;
            return false;
        }

        CLLNode* head = last->next;

        // Case 1: Only one node in circular list
        if (head == last && head->data.announcementId == id) {
            delete head;
            last = nullptr;
            std::cout << Color::GREEN << " [Success] Sole announcement #" << id << " deleted. Playlist is now empty.\n" << Color::RESET;
            return true;
        }

        // Case 2: Deleting the head node
        if (head->data.announcementId == id) {
            last->next = head->next;
            delete head;
            std::cout << Color::GREEN << " [Success] First announcement #" << id << " deleted.\n" << Color::RESET;
            return true;
        }

        // Case 3: Deleting an intermediate or last node
        CLLNode* prev = head;
```

```
        CLLNode* curr = head->next;
        while (curr != head) {
            if (curr->data.announcementId == id) {
                prev->next = curr->next;
                if (curr == last) {
                    last = prev; // Update tail pointer
                }
                delete curr;
                std::cout << Color::GREEN << " [Success] Announcement #" << id << " deleted.\n" << Color::RESET;
                return true;
            }
            prev = curr;
            curr = curr->next;
        }

        std::cout << Color::RED << " [Error] Announcement ID " << id << " not found in playlist.\n" << Color::RESET;
        return false;
    }

    // Search announcement
    bool searchAnnouncement(int id) const {
        if (!last) return false;
        CLLNode* curr = last->next;
        do {
            if (curr->data.announcementId == id) {
                std::cout << Color::GREEN << " [Found] #" << curr->data.announcementId
                          << ": \"" << curr->data.title << "\"\n" << Color::RESET;
                return true;
            }
            curr = curr->next;
        } while (curr != last->next);
        std::cout << Color::RED << " [Not Found] Announcement ID " << id << " is not in playlist.\n" << Color::RESET;
        return false;
    }

    // Display all announcements once
    void displayOnce() const {
        if (!last) {
            std::cout << Color::YELLOW << " [Notice] Playlist is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Campus Announcements (Single Pass) ---" << Color::RESET << "\n";
        CLLNode* curr = last->next;
        int count = 1;
        do {
            std::cout << " " << count++ << ". [ID: " << curr->data.announcementId << "] " << curr->data.title << "\n";
            curr = curr->next;
        } while (curr != last->next);
    }

    // Cyclic Traversal: Simulate continuous looping (e.g. 2 full cycles)
    void displayCyclic(int cycles = 2) const {
        if (!last) {
            std::cout << Color::YELLOW << " [Notice] Playlist is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::MAGENTA << Color::BOLD
                  << "--- Cyclic Broadcast Simulation (" << cycles << " Full Loops) ---" << Color::RESET << "\n";

        CLLNode* curr = last->next;
        int loopCount = 0;
        int step = 1;

        while (loopCount < cycles) {
            std::cout << " Step " << std::setw(2) << step++ << ": Broadcasting ["
                      << curr->data.announcementId << "] \"" << curr->data.title << "\"";
            if (curr == last) {
                loopCount++;
                std::cout << Color::YELLOW << " [Finished Loop " << loopCount << " -> Looping back to Start]" << Color::RESET;
            }
            std::cout << "\n";
            curr = curr->next;
        }
    }
};

// ============================================================================
// Module 4 Manager & Menu Handler
// ============================================================================
class CampusServicesManager {
private:
    SinglyLinkedList maintenanceList;
    DoublyLinkedList shuttleRoute;
    CircularLinkedList announcementPlaylist;

public:
    CampusServicesManager() {
        loadSampleData();
    }

    void loadSampleData() {
        // Sample Maintenance Requests
        maintenanceList.addRequest(MaintenanceRequest(101, "Prof. Sharma", "Room 302, IT Dept", "Projector bulb replacement", "Pending"));
        maintenanceList.addRequest(MaintenanceRequest(102, "Dr. Kulkarni", "Central Library", "AC unit temperature malfunction", "In Progress"));
        maintenanceList.addRequest(MaintenanceRequest(103, "Aditi Roy (Student)", "Girls Hostel B", "Wi-Fi access point reboot required", "Pending"));
        maintenanceList.addRequest(MaintenanceRequest(104, "Security Desk", "Main Gate", "Barrier arm sensor calibration", "Completed"));

        // Sample Shuttle Route Stations
        shuttleRoute.insertAtEnd(ShuttleStation(1, "Main Campus Gate"));
        shuttleRoute.insertAtEnd(ShuttleStation(2, "Administrative Building"));
        shuttleRoute.insertAtEnd(ShuttleStation(3, "Central Library & IT Dept"));
        shuttleRoute.insertAtEnd(ShuttleStation(4, "Auditorium & Sports Arena"));
        shuttleRoute.insertAtEnd(ShuttleStation(5, "Student Hostels & Mess"));

        // Sample Announcements
        announcementPlaylist.addAnnouncement(Announcement(1, "Annual Smart Campus Hackathon registrations close tonight!"));
        announcementPlaylist.addAnnouncement(Announcement(2, "Blood Donation Camp tomorrow at Central Auditorium (9 AM - 4 PM)"));
        announcementPlaylist.addAnnouncement(Announcement(3, "Mid-Semester exam timetable published on college portal"));
        announcementPlaylist.addAnnouncement(Announcement(4, "Guest lecture on Distributed Systems by Dr. Rao this Friday"));
    }

    void runMenu() {
```

```
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "═══════════════════════════════════════════════════\n"
                      << "    MODULE 4: CAMPUS SERVICES USING LINKED LISTS    \n"
                      << "═══════════════════════════════════════════════════\n"
                      << Color::RESET;
            std::cout << " [PART A: Singly Linked List - Maintenance Requests]\n"
                      << "   1. Add Maintenance Request\n"
                      << "   2. Display All Maintenance Requests\n"
                      << "   3. Update Request Status\n"
                      << "   4. Delete Request by ID\n"
                      << "   5. Count Pending Requests\n"
                      << " [PART B: Doubly Linked List - Campus Shuttle Route]\n"
                      << "   6. Insert Shuttle Station (Beginning / End / Position)\n"
                      << "   7. Delete Station by ID\n"
                      << "   8. Display Route Forward & Reverse\n"
                      << " [PART C: Circular Linked List - Announcements]\n"
                      << "   9. Add Announcement\n"
                      << "  10. Delete Announcement\n"
                      << "  11. Display Announcements (Single Pass & Cyclic Broadcast)\n"
                      << " [PART D: Complexity Comparison]\n"
                      << "  12. Singly vs Doubly vs Circular Linked Lists Comparison\n"
                      << "   0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-12): ");
            if (choice == 0) break;

            switch (choice) {
                case 1: {
                    int id = InputHelper::getInteger(" Enter Request ID: ");
                    std::string name = InputHelper::getString(" Enter Requester Name: ");
                    std::string loc = InputHelper::getString(" Enter Campus Location: ");
                    std::string desc = InputHelper::getString(" Enter Issue Description: ");
                    maintenanceList.addRequest(MaintenanceRequest(id, name, loc, desc, "Pending"));
                    break;
                }
                case 2:
                    maintenanceList.displayAll();
                    break;
                case 3: {
                    int id = InputHelper::getInteger(" Enter Request ID to update: ");
                    std::cout << " Choose Status: 1. Pending  2. In Progress  3. Completed\n";
                    int sChoice = InputHelper::getInteger(" Select status (1-3): ");
                    std::string statusStr = "Pending";
                    if (sChoice == 2) statusStr = "In Progress";
                    else if (sChoice == 3) statusStr = "Completed";
                    maintenanceList.updateStatus(id, statusStr);
                    break;
                }
                case 4: {
                    int id = InputHelper::getInteger(" Enter Request ID to delete: ");
                    maintenanceList.deleteRequest(id);
                    break;
                }
                case 5:
                    std::cout << Color::GREEN << " Total Pending Maintenance Requests: "
                              << maintenanceList.countPending() << Color::RESET << "\n";
                    break;
                case 6: {
                    std::cout << "\n Station Insertion: 1. Beginning  2. End  3. Specific Position\n";
                    int posChoice = InputHelper::getInteger(" Select insertion mode (1-3): ");
                    int id = InputHelper::getInteger(" Enter Station ID: ");
                    std::string name = InputHelper::getString(" Enter Station Name: ");
                    if (posChoice == 1) {
                        shuttleRoute.insertAtBeginning(ShuttleStation(id, name));
                    } else if (posChoice == 2) {
                        shuttleRoute.insertAtEnd(ShuttleStation(id, name));
                    } else {
                        int pos = InputHelper::getInteger(" Enter 1-based position: ");
                        shuttleRoute.insertAtPosition(ShuttleStation(id, name), pos);
                    }
                    break;
                }
                case 7: {
                    int id = InputHelper::getInteger(" Enter Station ID to delete: ");
                    shuttleRoute.deleteStation(id);
                    break;
                }
                case 8:
                    shuttleRoute.displayForward();
                    shuttleRoute.displayReverse();
                    break;
                case 9: {
                    int id = InputHelper::getInteger(" Enter Announcement ID: ");
                    std::string title = InputHelper::getString(" Enter Announcement Title: ");
                    announcementPlaylist.addAnnouncement(Announcement(id, title));
                    break;
                }
                case 10: {
                    int id = InputHelper::getInteger(" Enter Announcement ID to delete: ");
                    announcementPlaylist.deleteAnnouncement(id);
                    break;
                }
                case 11:
                    announcementPlaylist.displayOnce();
                    announcementPlaylist.displayCyclic(2);
                    break;
                case 12: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << "       LINKED LIST VARIATIONS: ARCHITECTURAL & COMPLEXITY COMPARISON                    \n"
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << Color::RESET
                              << " Feature               | Singly Linked List (SLL) | Doubly Linked List (DLL) | Circular Linked List (CLL)\n"
                              << "-------------------------+--------------------------+--------------------------+--------------------------\n"
                              << " Node Pointers         | 1 (next)                 | 2 (prev, next)           | 1 (next, loops to head)\n"
                              << " Memory Overhead       | Minimal (1 ptr/node)     | Highest (2 ptrs/node)    | Minimal (1 ptr/node)\n"
                              << " Traversal Direction   | Forward only             | Bidirectional (Fwd & Rev)| Cyclic (Continuous loop)\n"
                              << " Insert at Head        | O(1)                     | O(1)                     | O(1) (with tail pointer)\n"
                              << " Insert at Tail        | O(N) or O(1) with tail   | O(1) with tail           | O(1) (with tail pointer)\n"
                              << " Delete Given Node*    | O(N) (needs prev node)   | O(1) (via node->prev)    | O(N) (needs prev node)\n"
                              << " Reversal Complexity   | O(N)                     | O(1) (read via tail)     | O(N)\n"
```

```
                                << " Real-World Campus Use | Linear Task History      | Shuttle Two-way Route    | Continuous Digital Signage\n"
                                << "=============================================================================================\n";
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 12.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE4_LINKED_LISTS_HPP
```

```cpp
#ifndef MODULE5_STACKS_QUEUES_HPP
#define MODULE5_STACKS_QUEUES_HPP

#include "Common.hpp"
#include <algorithm>
#include <cctype>

// ============================================================================
// PART A: CUSTOM STACK FOR STUDENT RECORD UNDO FUNCTIONALITY
// ============================================================================
enum ActionType { ACTION_INSERT, ACTION_DELETE };

struct UndoRecord {
    ActionType action;
    Student studentData;

    UndoRecord(ActionType act, const Student& s) : action(act), studentData(s) {}
};

// Node for dynamic linked-stack implementation
struct StackNode {
    UndoRecord record;
    StackNode* next;
    StackNode(const UndoRecord& rec) : record(rec), next(nullptr) {}
};

class UndoStack {
private:
    StackNode* topNode;
    int sizeCount;

public:
    UndoStack() : topNode(nullptr), sizeCount(0) {}
    ~UndoStack() { clear(); }

    void clear() {
        while (topNode) {
            StackNode* temp = topNode;
            topNode = topNode->next;
            delete temp;
        }
        sizeCount = 0;
    }

    bool isEmpty() const { return topNode == nullptr; }
    int size() const { return sizeCount; }

    // Push action onto undo stack
    void push(ActionType act, const Student& s) {
        StackNode* newNode = new StackNode(UndoRecord(act, s));
        newNode->next = topNode;
        topNode = newNode;
        sizeCount++;
    }

    // Pop and retrieve the most recent action
    bool pop(UndoRecord& outRecord) {
        if (isEmpty()) return false;
        StackNode* temp = topNode;
        outRecord = temp->record;
        topNode = topNode->next;
        delete temp;
        sizeCount--;
        return true;
    }

    // Perform Undo on the student dataset
    bool performUndo(std::vector<Student>& students) {
        if (isEmpty()) {
            std::cout << Color::YELLOW << " [Notice] Undo stack is empty! No actions to reverse.\n" << Color::RESET;
            return false;
        }

        UndoRecord lastAction(ACTION_INSERT, Student());
        pop(lastAction);

        if (lastAction.action == ACTION_INSERT) {
            // Reversing an INSERTION -> Delete the inserted student
            int idToRemove = lastAction.studentData.id;
            for (auto it = students.begin(); it != students.end(); ++it) {
                if (it->id == idToRemove) {
                    students.erase(it);
                    std::cout << Color::GREEN << " [UNDO SUCCESS] Reverted INSERTION of Student ID "
                              << idToRemove << " (" << lastAction.studentData.name << "). Record removed.\n" << Color::RESET;
                    return true;
                }
            }
        } else if (lastAction.action == ACTION_DELETE) {
            // Reversing a DELETION -> Restore the saved student record
            students.push_back(lastAction.studentData);
            std::cout << Color::GREEN << " [UNDO SUCCESS] Reverted DELETION of Student ID "
                      << lastAction.studentData.id << " (" << lastAction.studentData.name << "). Record restored!\n" << Color::RESET;
            return true;
        }
        return false;
    }
};

// ============================================================================
// PART B: INFIX TO POSTFIX AND PREFIX EXPRESSION CONVERTER (CUSTOM STACK)
// ============================================================================
template <typename T>
class CustomArrayStack {
private:
    T* data;
    int capacity;
```

```
    int topIdx;

public:
    CustomArrayStack(int cap = 200) : capacity(cap), topIdx(-1) {
        data = new T[capacity];
    }
    ~CustomArrayStack() {
        delete[] data;
    }

    bool isEmpty() const { return topIdx == -1; }
    bool isFull() const { return topIdx == capacity - 1; }

    bool push(const T& val) {
        if (isFull()) return false;
        data[++topIdx] = val;
        return true;
    }

    bool pop(T& val) {
        if (isEmpty()) return false;
        val = data[topIdx--];
        return true;
    }

    T top() const {
        if (isEmpty()) return T();
        return data[topIdx];
    }
};

class ExpressionConverter {
private:
    // Operator precedence: ^ has 3, * / has 2, + - has 1
    static int getPrecedence(char op) {
        if (op == '^') return 3;
        if (op == '*' || op == '/') return 2;
        if (op == '+' || op == '-') return 1;
        return 0;
    }

    // Associativity: '^' is Right-to-Left (true), others are Left-to-Right (false)
    static bool isRightAssociative(char op) {
        return (op == '^');
    }

    static bool isOperator(char ch) {
        return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');
    }

    static bool isOperand(char ch) {
        return std::isalnum(static_cast<unsigned char>(ch));
    }

public:
    // Validate matching parentheses and basic expression syntax
    static bool validateInfix(const std::string& expr, std::string& errorMsg) {
        CustomArrayStack<char> parenStack(100);
        int operandCount = 0;
        int operatorCount = 0;

        for (size_t i = 0; i < expr.length(); ++i) {
            char ch = expr[i];
            if (std::isspace(static_cast<unsigned char>(ch))) continue;

            if (ch == '(') {
                parenStack.push(ch);
            } else if (ch == ')') {
                char dummy;
                if (!parenStack.pop(dummy)) {
                    errorMsg = "Mismatched closing parenthesis ')' at position " + std::to_string(i + 1);
                    return false;
                }
            } else if (isOperator(ch)) {
                operatorCount++;
            } else if (isOperand(ch)) {
                operandCount++;
            } else {
                errorMsg = "Invalid character '" + std::string(1, ch) + "' in expression";
                return false;
            }
        }

        if (!parenStack.isEmpty()) {
            errorMsg = "Mismatched opening parenthesis '(' without closing pair";
            return false;
        }

        if (operandCount == 0) {
            errorMsg = "Expression contains no operands";
            return false;
        }

        return true;
    }

    // Convert Infix -> Postfix
    static std::string infixToPostfix(const std::string& infix) {
        CustomArrayStack<char> opStack(200);
        std::string postfix = "";

        for (char ch : infix) {
            if (std::isspace(static_cast<unsigned char>(ch))) continue;

            if (isOperand(ch)) {
                postfix += ch;
            } else if (ch == '(') {
                opStack.push(ch);
            } else if (ch == ')') {
                char popped;
                while (!opStack.isEmpty() && opStack.top() != '(') {
                    opStack.pop(popped);
```

```
                    postfix += popped;
                }
                opStack.pop(popped); // Remove '('
            } else if (isOperator(ch)) {
                while (!opStack.isEmpty() && opStack.top() != '(') {
                    char topOp = opStack.top();
                    int precTop = getPrecedence(topOp);
                    int precCurr = getPrecedence(ch);

                    if (precTop > precCurr || (precTop == precCurr && !isRightAssociative(ch))) {
                        char popped;
                        opStack.pop(popped);
                        postfix += popped;
                    } else {
                        break;
                    }
                }
                opStack.push(ch);
            }
        }

        while (!opStack.isEmpty()) {
            char popped;
            opStack.pop(popped);
            postfix += popped;
        }
        return postfix;
    }

    // Convert Infix --> Prefix
    // Algorithm:
    // 1. Reverse infix string (swapping '(' and ')')
    // 2. Obtain postfix expression with modified operator precedence check
    // 3. Reverse the resulting postfix expression --> Prefix!
    static std::string infixToPrefix(const std::string& infix) {
        std::string revInfix = "";
        for (int i = static_cast<int>(infix.length()) - 1; i >= 0; --i) {
            char ch = infix[i];
            if (ch == '(') revInfix += ')';
            else if (ch == ')') revInfix += '(';
            else revInfix += ch;
        }

        CustomArrayStack<char> opStack(200);
        std::string revPostfix = "";

        for (char ch : revInfix) {
            if (std::isspace(static_cast<unsigned char>(ch))) continue;

            if (isOperand(ch)) {
                revPostfix += ch;
            } else if (ch == '(') {
                opStack.push(ch);
            } else if (ch == ')') {
                char popped;
                while (!opStack.isEmpty() && opStack.top() != '(') {
                    opStack.pop(popped);
                    revPostfix += popped;
                }
                opStack.pop(popped); // Discard '('
            } else if (isOperator(ch)) {
                while (!opStack.isEmpty() && opStack.top() != '(') {
                    char topOp = opStack.top();
                    int precTop = getPrecedence(topOp);
                    int precCurr = getPrecedence(ch);

                    // For reversed expression: if operator is right-associative (^), pop on >=;
                    // if left-associative (+,-,*,/), pop only on strictly >
                    if (precTop > precCurr || (precTop == precCurr && isRightAssociative(ch))) {
                        char popped;
                        opStack.pop(popped);
                        revPostfix += popped;
                    } else {
                        break;
                    }
                }
                opStack.push(ch);
            }
        }

        while (!opStack.isEmpty()) {
            char popped;
            opStack.pop(popped);
            revPostfix += popped;
        }

        // Reverse the modified postfix string to obtain the true Prefix expression
        std::string prefix = revPostfix;
        std::reverse(prefix.begin(), prefix.end());
        return prefix;
    }
};

// ============================================================================
// PART C: ARRAY-BASED LINEAR QUEUE (Student Service Requests)
// ============================================================================
struct ServiceTicket {
    int ticketId;
    std::string studentName;
    std::string serviceType; // e.g., "Transcript Issue", "Bonafide Certificate"

    ServiceTicket(int id = 0, const std::string& name = "", const std::string& type = "")
        : ticketId(id), studentName(name), serviceType(type) {}
};

class LinearQueue {
private:
    ServiceTicket* arr;
    int capacity;
    int frontIdx;
    int rearIdx;

```

```
public:
    LinearQueue(int cap = 6) : capacity(cap), frontIdx(-1), rearIdx(-1) {
        arr = new ServiceTicket[capacity];
    }
    ~LinearQueue() { delete[] arr; }

    bool isEmpty() const {
        return (frontIdx == -1 || frontIdx > rearIdx);
    }

    bool isFull() const {
        return (rearIdx == capacity - 1);
    }

    // Enqueue at rear
    bool enqueue(const ServiceTicket& ticket) {
        if (isFull()) {
            std::cout << Color::RED << " [QUEUE OVERFLOW] Linear queue is full at index "
                      << rearIdx << "! Cannot enqueue Ticket #" << ticket.ticketId
                      << "\n (Notice: Linear queue cannot reuse freed slots at front without shifting)\n" << Color::RESET;
            return false;
        }
        if (frontIdx == -1) frontIdx = 0;
        arr[++rearIdx] = ticket;
        std::cout << Color::GREEN << " [Enqueued] Ticket #" << ticket.ticketId
                  << " added at rear position " << rearIdx << ".\n" << Color::RESET;
        return true;
    }

    // Dequeue from front
    bool dequeue(ServiceTicket& servedTicket) {
        if (isEmpty()) {
            std::cout << Color::RED << " [QUEUE UNDERFLOW] Linear queue is empty! No service requests to process.\n" << Color::RESET;
            return false;
        }
        servedTicket = arr[frontIdx++];
        std::cout << Color::GREEN << " [Processed] Ticket #" << servedTicket.ticketId
                  << " (" << servedTicket.studentName << " - " << servedTicket.serviceType
                  << ") processed from front position " << (frontIdx - 1) << ".\n" << Color::RESET;
        return true;
    }

    // Display all pending requests
    void display() const {
        if (isEmpty()) {
            std::cout << Color::YELLOW << " [Notice] Linear queue has no pending tickets.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "--- Pending Service Requests in Linear Queue (Front: " << frontIdx << ", Rear: " << rearIdx << ") ---"
                  << Color::RESET << "\n";
        for (int i = frontIdx; i <= rearIdx; ++i) {
            std::cout << " Slot [" << i << "] -> Ticket #" << arr[i].ticketId
                      << " | " << arr[i].studentName
                      << " | Service: " << arr[i].serviceType << "\n";
        }
    }

    // Search by ID
    bool searchById(int id) const {
        if (isEmpty()) return false;
        for (int i = frontIdx; i <= rearIdx; ++i) {
            if (arr[i].ticketId == id) {
                std::cout << Color::GREEN << " [Found] Ticket #" << id
                          << " found at slot " << i << " (" << arr[i].studentName << ").\n" << Color::RESET;
                return true;
            }
        }
        std::cout << Color::RED << " [Not Found] Ticket #" << id << " not in pending queue.\n" << Color::RESET;
        return false;
    }
};

// ============================================================================
// PART D: ARRAY-BASED CIRCULAR QUEUE (Recurring Campus Service Scheduling)
// ============================================================================
class CircularQueue {
private:
    ServiceTicket* arr;
    int capacity;
    int front;
    int rear;

public:
    CircularQueue(int cap = 6) : capacity(cap), front(-1), rear(-1) {
        arr = new ServiceTicket[capacity];
    }
    ~CircularQueue() { delete[] arr; }

    bool isEmpty() const {
        return (front == -1);
    }

    bool isFull() const {
        return ((rear + 1) % capacity == front);
    }

    // Enqueue in Circular Queue
    bool enqueue(const ServiceTicket& item) {
        if (isFull()) {
            std::cout << Color::RED << " [CIRCULAR QUEUE FULL] Capacity (" << capacity << ") reached! Cannot add Ticket #"
                      << item.ticketId << ".\n" << Color::RESET;
            return false;
        }

        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % capacity;
        }

        arr[rear] = item;
```

```
        std::cout << Color::GREEN << " [Enqueued] Ticket #" << item.ticketId
                  << " entered circular queue at physical index " << rear << ".\n" << Color::RESET;
        return true;
    }

    // Dequeue in Circular Queue
    bool dequeue(ServiceTicket& item) {
        if (isEmpty()) {
            std::cout << Color::RED << " [CIRCULAR QUEUE EMPTY] No requests available.\n" << Color::RESET;
            return false;
        }

        item = arr[front];
        std::cout << Color::GREEN << " [Dispatched] Ticket #" << item.ticketId
                  << " processed from physical index " << front << ".\n" << Color::RESET;

        if (front == rear) {
            // Queue had only one element, now reset to empty
            front = rear = -1;
        } else {
            front = (front + 1) % capacity;
        }
        return true;
    }

    // Display pending requests in logical circular sequence
    void display() const {
        if (isEmpty()) {
            std::cout << Color::YELLOW << " [Notice] Circular queue is empty.\n" << Color::RESET;
            return;
        }

        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "--- Recurring Service Circular Queue (Front: " << front << ", Rear: " << rear << ") ---"
                  << Color::RESET << "\n";

        int i = front;
        int count = 1;
        while (true) {
            std::cout << " " << count++ << ". [Array Pos " << i << "] Ticket #" << arr[i].ticketId
                      << " | Student: " << arr[i].studentName
                      << " | Type: " << arr[i].serviceType << "\n";
            if (i == rear) break;
            i = (i + 1) % capacity;
        }
    }
};

// ============================================================================
// Module 5 Manager & Menu Handler
// ============================================================================
class StackQueueManager {
private:
    UndoStack undoHistory;
    LinearQueue serviceQueue;
    CircularQueue circularSchedule;

public:
    StackQueueManager() : serviceQueue(5), circularSchedule(5) {
        // Pre-fill sample tickets
        serviceQueue.enqueue(ServiceTicket(201, "Neha Patil", "Hostel Leave Approval"));
        serviceQueue.enqueue(ServiceTicket(202, "Amit Joshi", "Identity Card Duplicate"));
        serviceQueue.enqueue(ServiceTicket(203, "Kavya Nair", "Fee Clearance Receipt"));

        circularSchedule.enqueue(ServiceTicket(301, "Campus Security", "Night Patrol Sweep A"));
        circularSchedule.enqueue(ServiceTicket(302, "Facility Team", "Library Sanitization"));
        circularSchedule.enqueue(ServiceTicket(303, "Sports Desk", "Gymnasium Equipment Inspection"));
    }

    UndoStack& getUndoStack() { return undoHistory; }

    void runMenu(std::vector<Student>& sharedStudents) {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "============================================================\n"
                      << "   MODULE 5: STACKS, QUEUES & EXPRESSION CONVERSION     \n"
                      << "============================================================\n"
                      << Color::RESET;
            std::cout << " [PART A: Stack - Student Record Undo]\n"
                      << "   1. Undo Most Recent Student Modification (" << undoHistory.size() << " action(s) in stack)\n"
                      << "   2. Display Current Student Records\n"
                      << " [PART B: Stack - Infix Expression Conversion]\n"
                      << "   3. Convert Infix to Postfix and Prefix\n"
                      << "   4. Test Standard Example (A+B*C)\n"
                      << " [PART C: Linear Queue - Student Service Desk]\n"
                      << "   5. Enqueue Student Service Ticket (Linear)\n"
                      << "   6. Process Service Ticket (Linear Dequeue)\n"
                      << "   7. Display Linear Queue Tickets\n"
                      << "   8. Search Ticket by ID in Linear Queue\n"
                      << " [PART D: Circular Queue - Recurring Campus Services]\n"
                      << "   9. Enqueue Recurring Task (Circular)\n"
                      << "  10. Process Next Task (Circular Dequeue)\n"
                      << "  11. Display Circular Queue in Logical Sequence\n"
                      << " [PART E: Complexity Comparison]\n"
                      << "  12. LIFO vs FIFO & Linear vs Circular Queue Analysis\n"
                      << "   0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-12): ");
            if (choice == 0) break;

            switch (choice) {
                case 1: {
                    undoHistory.performUndo(sharedStudents);
                    break;
                }
                case 2: {
                    std::cout << "\n" << Color::CYAN << Color::BOLD
                              << "+--------+----------------------+------+----------------+---------+-------------------------+\n"
                              << "| ID     | Name                 | Div  | Contact        | Average | Marks (M, P, C, CS) |\n"
                              << "+--------+----------------------+------+----------------+---------+-------------------------+\n"
                              << Color::RESET;
                    for (const auto& s : sharedStudents) s.printRow();
```

```
                    std::cout << Color::CYAN << "+--------+--------------------+------+--------------+----------+----------------------+\n" << Color::RESET;
                    break;
                }
                case 3: {
                    std::string infix = InputHelper::getString(" Enter valid Infix expression (e.g. (A+B)*(C-D)^E): ");
                    std::string errorMsg;
                    if (!ExpressionConverter::validateInfix(infix, errorMsg)) {
                        std::cout << Color::RED << " [Syntax Error] " << errorMsg << "\n" << Color::RESET;
                    } else {
                        std::string postfix = ExpressionConverter::infixToPostfix(infix);
                        std::string prefix = ExpressionConverter::infixToPrefix(infix);
                        std::cout << "\n" << Color::GREEN << Color::BOLD << " Expression Results:" << Color::RESET << "\n"
                                  << "   Infix:   " << Color::CYAN << infix << Color::RESET << "\n"
                                  << "   Postfix: " << Color::YELLOW << postfix << Color::RESET << "\n"
                                  << "   Prefix:  " << Color::MAGENTA << prefix << Color::RESET << "\n";
                    }
                    break;
                }
                case 4: {
                    std::string testExpr = "A+B*C";
                    std::string postfix = ExpressionConverter::infixToPostfix(testExpr);
                    std::string prefix = ExpressionConverter::infixToPrefix(testExpr);
                    std::cout << "\n" << Color::GREEN << Color::BOLD << " Verified Specification Benchmark:" << Color::RESET << "\n"
                              << "   Infix:   " << Color::CYAN << testExpr << Color::RESET << "\n"
                              << "   Postfix: " << Color::YELLOW << postfix << Color::RESET << " (Expected: ABC*+)\n"
                              << "   Prefix:  " << Color::MAGENTA << prefix << Color::RESET << " (Expected: +A*BC)\n";
                    break;
                }
                case 5: {
                    int id = InputHelper::getInteger(" Enter Ticket ID: ");
                    std::string name = InputHelper::getString(" Enter Student Name: ");
                    std::string type = InputHelper::getString(" Enter Service Requested: ");
                    serviceQueue.enqueue(ServiceTicket(id, name, type));
                    break;
                }
                case 6: {
                    ServiceTicket served;
                    serviceQueue.dequeue(served);
                    break;
                }
                case 7:
                    serviceQueue.display();
                    break;
                case 8: {
                    int id = InputHelper::getInteger(" Enter Ticket ID to search: ");
                    serviceQueue.searchById(id);
                    break;
                }
                case 9: {
                    int id = InputHelper::getInteger(" Enter Task ID: ");
                    std::string dept = InputHelper::getString(" Enter Facility/Team: ");
                    std::string desc = InputHelper::getString(" Enter Recurring Task Description: ");
                    circularSchedule.enqueue(ServiceTicket(id, dept, desc));
                    break;
                }
                case 10: {
                    ServiceTicket task;
                    circularSchedule.dequeue(task);
                    break;
                }
                case 11:
                    circularSchedule.display();
                    break;
                case 12: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "================================================================================================\n"
                              << "          THEORETICAL ANALYSIS: LIFO vs FIFO & QUEUE ARCHITECTURES                              \n"
                              << "================================================================================================\n"
                              << Color::RESET
                              << " 1. LIFO (Last-In, First-Out) - Stacks:\n"
                              << "    - Operations: Push O(1), Pop O(1), Top O(1).\n"
                              << "    - Applications: Function call stacks, Expression parsing, Reverse/Undo chains.\n\n"
                              << " 2. FIFO (First-In, First-Out) - Queues:\n"
                              << "    - Operations: Enqueue O(1), Dequeue O(1).\n"
                              << "    - Applications: CPU process scheduling, Printer spoolers, Student ticket processing.\n\n"
                              << " 3. LINEAR QUEUE vs CIRCULAR QUEUE COMPARISON:\n"
                              << "    - Linear Queue Limitation: 'False Overflow'. When rear reaches max index, no more\n"
                              << "      items can be added even if front has moved ahead, leaving empty slots wasted.\n"
                              << "    - Circular Queue Solution: Uses modulo arithmetic ((rear + 1) % MAX) to wrap around\n"
                              << "      and reuse vacated memory cells at the beginning of the buffer.\n"
                              << "    - Full Condition: (rear + 1) % Capacity == front\n"
                              << "    - Empty Condition: front == -1\n"
                              << "================================================================================================\n";
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 12.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE5_STACKS_QUEUES_HPP
```

```cpp
#ifndef MODULE6_TREES_HPP
#define MODULE6_TREES_HPP

#include "Common.hpp"
#include <queue>
#include <algorithm>

// ============================================================================
// PART A: BINARY SEARCH TREE (BST)
// ============================================================================
struct BSTNode {
    int studentId;
    Student data;
    BSTNode* left;
    BSTNode* right;

    BSTNode(const Student& s) : studentId(s.id), data(s), left(nullptr), right(nullptr) {}
};

class BinarySearchTree {
private:
    BSTNode* root;

    // Helper: Recursive Insertion
    BSTNode* insertHelper(BSTNode* node, const Student& s, bool& inserted) {
        if (!node) {
            inserted = true;
            return new BSTNode(s);
        }
        if (s.id == node->studentId) {
            inserted = false; // Duplicate rejected
            return node;
        } else if (s.id < node->studentId) {
            node->left = insertHelper(node->left, s, inserted);
        } else {
            node->right = insertHelper(node->right, s, inserted);
        }
        return node;
    }

    // Helper: Find minimum node (Inorder Successor)
    BSTNode* findMin(BSTNode* node) const {
        while (node && node->left) node = node->left;
        return node;
    }

    // Helper: Recursive Deletion (Handles Leaf, 1-Child, 2-Children)
    BSTNode* deleteHelper(BSTNode* node, int id, bool& deleted) {
        if (!node) return nullptr;

        if (id < node->studentId) {
            node->left = deleteHelper(node->left, id, deleted);
        } else if (id > node->studentId) {
            node->right = deleteHelper(node->right, id, deleted);
        } else {
            // Node found!
            deleted = true;
            // Case 1 & 2: Leaf or Single Child
            if (!node->left) {
                BSTNode* temp = node->right;
                delete node;
                return temp;
            } else if (!node->right) {
                BSTNode* temp = node->left;
                delete node;
                return temp;
            }
            // Case 3: Two Children -> Replace with Inorder Successor
            BSTNode* successor = findMin(node->right);
            node->studentId = successor->studentId;
            node->data = successor->data;
            node->right = deleteHelper(node->right, successor->studentId, deleted);
        }
        return node;
    }

    // Helper: Recursive Search
    BSTNode* searchHelper(BSTNode* node, int id, int& comparisons) const {
        if (!node) return nullptr;
        comparisons++;
        if (node->studentId == id) return node;
        if (id < node->studentId) return searchHelper(node->left, id, comparisons);
        return searchHelper(node->right, id, comparisons);
    }

    // Traversals
    void inorderHelper(BSTNode* node) const {
        if (!node) return;
        inorderHelper(node->left);
        std::cout << "[" << node->studentId << ": " << node->data.name << "] ";
        inorderHelper(node->right);
    }

    void preorderHelper(BSTNode* node) const {
        if (!node) return;
        std::cout << "[" << node->studentId << ": " << node->data.name << "] ";
        preorderHelper(node->left);
        preorderHelper(node->right);
    }

    void postorderHelper(BSTNode* node) const {
        if (!node) return;
        postorderHelper(node->left);
        postorderHelper(node->right);
        std::cout << "[" << node->studentId << ": " << node->data.name << "] ";
    }
```

```
    // Tree Height
    int heightHelper(BSTNode* node) const {
        if (!node) return 0;
        return 1 + std::max(heightHelper(node->left), heightHelper(node->right));
    }

    // Deep Copy helper
    BSTNode* copyHelper(BSTNode* node) const {
        if (!node) return nullptr;
        BSTNode* newNode = new BSTNode(node->data);
        newNode->left = copyHelper(node->left);
        newNode->right = copyHelper(node->right);
        return newNode;
    }

    // Mirror Image helper (swaps left and right recursively)
    BSTNode* mirrorHelper(BSTNode* node) const {
        if (!node) return nullptr;
        BSTNode* mirrored = new BSTNode(node->data);
        mirrored->left = mirrorHelper(node->right);
        mirrored->right = mirrorHelper(node->left);
        return mirrored;
    }

    // Parents with Children
    void printParentsWithChildren(BSTNode* node) const {
        if (!node) return;
        if (node->left || node->right) {
            std::cout << " Parent [" << node->studentId << ": " << node->data.name << "] -> ";
            if (node->left) std::cout << "Left: [" << node->left->studentId << "] ";
            else std::cout << "Left: [None] ";
            if (node->right) std::cout << "Right: [" << node->right->studentId << "]";
            else std::cout << "Right: [None]";
            std::cout << "\n";
        }
        printParentsWithChildren(node->left);
        printParentsWithChildren(node->right);
    }

    // Leaf nodes
    void printLeaves(BSTNode* node, int& count) const {
        if (!node) return;
        if (!node->left && !node->right) {
            std::cout << " Leaf Node: [" << node->studentId << ": " << node->data.name << "]\n";
            count++;
            return;
        }
        printLeaves(node->left, count);
        printLeaves(node->right, count);
    }

    // Memory clean up
    void destroy(BSTNode* node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }

public:
    BinarySearchTree() : root(nullptr) {}
    ~BinarySearchTree() { destroy(root); }

    bool insert(const Student& s) {
        bool inserted = false;
        root = insertHelper(root, s, inserted);
        return inserted;
    }

    bool remove(int id) {
        bool deleted = false;
        root = deleteHelper(root, id, deleted);
        return deleted;
    }

    BSTNode* search(int id, int& comparisons) const {
        comparisons = 0;
        return searchHelper(root, id, comparisons);
    }

    void inorder() const {
        std::cout << " Inorder (Sorted L-Root-R):   ";
        inorderHelper(root);
        std::cout << "\n";
    }

    void preorder() const {
        std::cout << " Preorder (Root-L-R):          ";
        preorderHelper(root);
        std::cout << "\n";
    }

    void postorder() const {
        std::cout << " Postorder (L-R-Root):         ";
        postorderHelper(root);
        std::cout << "\n";
    }

    int getHeight() const {
        return heightHelper(root);
    }

    void displayParentsAndChildren() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] BST is empty.\n" << Color::RESET;
            return;
        }
        std::cout << Color::CYAN << "--- BST Parent Nodes & Their Respective Children ---" << Color::RESET << "\n";
        printParentsWithChildren(root);
    }

```

```
    void displayLeaves() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] BST is empty.\n" << Color::RESET;
            return;
        }
        std::cout << Color::CYAN << "--- BST Leaf Nodes (External Nodes) ---" << Color::RESET << "\n";
        int count = 0;
        printLeaves(root, count);
        std::cout << " Total Leaf Nodes: " << count << "\n";
    }

    // Level-Order Traversal using Queue
    void levelOrder() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] BST is empty.\n" << Color::RESET;
            return;
        }
        std::cout << Color::CYAN << "--- BST Level-Order (Breadth-First) Traversal ---" << Color::RESET << "\n";
        std::queue<BSTNode*> q;
        q.push(root);
        int level = 0;

        while (!q.empty()) {
            int levelSize = static_cast<int>(q.size());
            std::cout << " Level " << level++ << ": ";
            for (int i = 0; i < levelSize; ++i) {
                BSTNode* curr = q.front();
                q.pop();
                std::cout << "[" << curr->studentId << "] ";
                if (curr->left) q.push(curr->left);
                if (curr->right) q.push(curr->right);
            }
            std::cout << "\n";
        }
    }

    // Mirror image creation and traversal
    void displayMirror() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] BST is empty.\n" << Color::RESET;
            return;
        }
        BSTNode* mirroredRoot = mirrorHelper(root);
        std::cout << Color::MAGENTA << " Mirror Image Inorder Traversal (R-Root-L descending): " << Color::RESET;
        inorderHelper(mirroredRoot);
        std::cout << "\n";
        destroy(mirroredRoot); // Clean temporary mirror copy
    }

    // Deep independent copy verification
    void verifyDeepCopy() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] BST is empty.\n" << Color::RESET;
            return;
        }
        BSTNode* copyRoot = copyHelper(root);
        std::cout << Color::GREEN << " Independent copy created successfully in separate heap memory!\n"
                  << " Original Root Address: " << root << " | Copy Root Address: " << copyRoot << "\n"
                  << " Copy Inorder Traversal: ";
        inorderHelper(copyRoot);
        std::cout << "\n" << Color::RESET;
        destroy(copyRoot);
    }
};

// ============================================================================
// PART B: AVL TREE (SELF-BALANCING BINARY SEARCH TREE)
// ============================================================================
struct AVLNode {
    int studentId;
    Student data;
    int height;
    AVLNode* left;
    AVLNode* right;

    AVLNode(const Student& s) : studentId(s.id), data(s), height(1), left(nullptr), right(nullptr) {}
};

class AVLTree {
private:
    AVLNode* root;

    int getHeight(AVLNode* node) const {
        return node ? node->height : 0;
    }

    int getBalanceFactor(AVLNode* node) const {
        return node ? (getHeight(node->left) - getHeight(node->right)) : 0;
    }

    // LL Case: Right Rotation
    AVLNode* rightRotate(AVLNode* y) {
        AVLNode* x = y->left;
        AVLNode* T2 = x->right;

        // Perform rotation
        x->right = y;
        y->left = T2;

        // Update heights
        y->height = 1 + std::max(getHeight(y->left), getHeight(y->right));
        x->height = 1 + std::max(getHeight(x->left), getHeight(x->right));

        return x; // New root
    }

    // RR Case: Left Rotation
    AVLNode* leftRotate(AVLNode* x) {
        AVLNode* y = x->right;
        AVLNode* T2 = y->left;

        // Perform rotation
```

```
        y->left = x;
        x->right = T2;

        // Update heights
        x->height = 1 + std::max(getHeight(x->left), getHeight(x->right));
        y->height = 1 + std::max(getHeight(y->left), getHeight(y->right));

        return y; // New root
    }

    // Recursive AVL Insertion with Automatic Rotations
    AVLNode* insertHelper(AVLNode* node, const Student& s, bool& inserted) {
        if (!node) {
            inserted = true;
            return new AVLNode(s);
        }

        if (s.id < node->studentId) {
            node->left = insertHelper(node->left, s, inserted);
        } else if (s.id > node->studentId) {
            node->right = insertHelper(node->right, s, inserted);
        } else {
            inserted = false; // Duplicates rejected
            return node;
        }

        // Update node height
        node->height = 1 + std::max(getHeight(node->left), getHeight(node->right));

        // Compute Balance Factor to check if node became unbalanced
        int balance = getBalanceFactor(node);

        // 4 Balancing Rotation Cases:
        // Case 1: Left-Left (LL) -> Right Rotation
        if (balance > 1 && s.id < node->left->studentId) {
            return rightRotate(node);
        }
        // Case 2: Right-Right (RR) -> Left Rotation
        if (balance < -1 && s.id > node->right->studentId) {
            return leftRotate(node);
        }
        // Case 3: Left-Right (LR) -> Left Rotate left child, then Right Rotate node
        if (balance > 1 && s.id > node->left->studentId) {
            node->left = leftRotate(node->left);
            return rightRotate(node);
        }
        // Case 4: Right-Left (RL) -> Right Rotate right child, then Left Rotate node
        if (balance < -1 && s.id < node->right->studentId) {
            node->right = rightRotate(node->right);
            return leftRotate(node);
        }

        return node;
    }

    AVLNode* findMin(AVLNode* node) const {
        while (node && node->left) node = node->left;
        return node;
    }

    // Recursive AVL Deletion with Automatic Rebalancing
    AVLNode* deleteHelper(AVLNode* node, int id, bool& deleted) {
        if (!node) return nullptr;

        if (id < node->studentId) {
            node->left = deleteHelper(node->left, id, deleted);
        } else if (id > node->studentId) {
            node->right = deleteHelper(node->right, id, deleted);
        } else {
            deleted = true;
            if (!node->left || !node->right) {
                AVLNode* temp = node->left ? node->left : node->right;
                if (!temp) {
                    temp = node;
                    node = nullptr;
                } else {
                    *node = *temp;
                }
                delete temp;
            } else {
                AVLNode* temp = findMin(node->right);
                node->studentId = temp->studentId;
                node->data = temp->data;
                node->right = deleteHelper(node->right, temp->studentId, deleted);
            }
        }

        if (!node) return nullptr;

        // Update height
        node->height = 1 + std::max(getHeight(node->left), getHeight(node->right));

        // Rebalance
        int balance = getBalanceFactor(node);

        if (balance > 1 && getBalanceFactor(node->left) >= 0) {
            return rightRotate(node);
        }
        if (balance > 1 && getBalanceFactor(node->left) < 0) {
            node->left = leftRotate(node->left);
            return rightRotate(node);
        }
        if (balance < -1 && getBalanceFactor(node->right) <= 0) {
            return leftRotate(node);
        }
        if (balance < -1 && getBalanceFactor(node->right) > 0) {
            node->right = rightRotate(node->right);
            return leftRotate(node);
        }

        return node;
    }
```

```
    void inorderDetailed(AVLNode* node) const {
        if (!node) return;
        inorderDetailed(node->left);
        std::cout << " [ID: " << std::setw(4) << node->studentId
                  << " | " << std::setw(15) << std::left << node->data.name.substr(0, 15) << std::right
                  << " | H: " << node->height
                  << " | BF: " << std::setw(2) << getBalanceFactor(node) << "]\n";
        inorderDetailed(node->right);
    }

    void destroy(AVLNode* node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }

public:
    AVLTree() : root(nullptr) {}
    ~AVLTree() { destroy(root); }

    bool insert(const Student& s) {
        bool inserted = false;
        root = insertHelper(root, s, inserted);
        return inserted;
    }

    bool remove(int id) {
        bool deleted = false;
        root = deleteHelper(root, id, deleted);
        return deleted;
    }

    void displayInorderWithBalance() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] AVL Tree is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "--- AVL Tree Inorder Traversal (Height & Balance Factors) ---" << Color::RESET << "\n";
        inorderDetailed(root);
        std::cout << " Overall AVL Tree Height: " << getHeight(root) << "\n";
    }

    AVLNode* search(int id, int& comparisons) const {
        comparisons = 0;
        AVLNode* curr = root;
        while (curr) {
            comparisons++;
            if (curr->studentId == id) return curr;
            if (id < curr->studentId) curr = curr->left;
            else curr = curr->right;
        }
        return nullptr;
    }
};

// ============================================================================
// PART C: B+ TREE (ORDER 3 HIERARCHICAL INDEXING FOR RANGE QUERIES)
// In a B+ Tree:
// - Internal nodes hold routing keys and child pointers.
// - Leaf nodes hold actual data records and a linked 'next' pointer.
// ============================================================================
const int BPLUS_ORDER = 3; // Max keys in node = 2, max children = 3

struct BPlusNode {
    bool isLeaf;
    int numKeys;
    int keys[BPLUS_ORDER];              // Space for overflow during split
    Student records[BPLUS_ORDER];       // Used in leaf nodes
    BPlusNode* children[BPLUS_ORDER + 1];// Used in internal nodes
    BPlusNode* next;                    // Linked leaf pointer for range queries

    BPlusNode(bool leaf = true) : isLeaf(leaf), numKeys(0), next(nullptr) {
        for (int i = 0; i <= BPLUS_ORDER; ++i) children[i] = nullptr;
    }
};

class BPlusTree {
private:
    BPlusNode* root;

    // Search leaf containing the key
    BPlusNode* findLeaf(BPlusNode* curr, int id) const {
        if (!curr) return nullptr;
        while (!curr->isLeaf) {
            int i = 0;
            while (i < curr->numKeys && id >= curr->keys[i]) {
                i++;
            }
            curr = curr->children[i];
        }
        return curr;
    }

    // Split child of an internal node
    void splitChild(BPlusNode* parent, int idx, BPlusNode* fullChild) {
        BPlusNode* rightSibling = new BPlusNode(fullChild->isLeaf);
        int mid = fullChild->numKeys / 2;

        if (fullChild->isLeaf) {
            // Leaf Split: keep mid element in right sibling, copy up to parent
            rightSibling->numKeys = fullChild->numKeys - mid;
            for (int j = 0; j < rightSibling->numKeys; ++j) {
                rightSibling->keys[j] = fullChild->keys[mid + j];
                rightSibling->records[j] = fullChild->records[mid + j];
            }
            fullChild->numKeys = mid;

            // Link leaves
            rightSibling->next = fullChild->next;
```

```
            fullChild->next = rightSibling;

            // Insert promoted key into parent
            for (int j = parent->numKeys; j > idx; --j) {
                parent->keys[j] = parent->keys[j - 1];
                parent->children[j + 1] = parent->children[j];
            }
            parent->keys[idx] = rightSibling->keys[0];
            parent->children[idx + 1] = rightSibling;
            parent->numKeys++;
        } else {
            // Internal Node Split: push mid element up to parent
            int promotedKey = fullChild->keys[mid];
            rightSibling->numKeys = fullChild->numKeys - mid - 1;

            for (int j = 0; j < rightSibling->numKeys; ++j) {
                rightSibling->keys[j] = fullChild->keys[mid + 1 + j];
                rightSibling->children[j] = fullChild->children[mid + 1 + j];
            }
            rightSibling->children[rightSibling->numKeys] = fullChild->children[fullChild->numKeys];
            fullChild->numKeys = mid;

            for (int j = parent->numKeys; j > idx; --j) {
                parent->keys[j] = parent->keys[j - 1];
                parent->children[j + 1] = parent->children[j];
            }
            parent->keys[idx] = promotedKey;
            parent->children[idx + 1] = rightSibling;
            parent->numKeys++;
        }
    }

    void insertNonFull(BPlusNode* node, const Student& s) {
        int i = node->numKeys - 1;

        if (node->isLeaf) {
            // Insert into sorted leaf
            while (i >= 0 && s.id < node->keys[i]) {
                node->keys[i + 1] = node->keys[i];
                node->records[i + 1] = node->records[i];
                i--;
            }
            node->keys[i + 1] = s.id;
            node->records[i + 1] = s;
            node->numKeys++;
        } else {
            // Find child to traverse
            while (i >= 0 && s.id < node->keys[i]) {
                i--;
            }
            i++;

            if (node->children[i]->numKeys == (BPLUS_ORDER - 1)) {
                splitChild(node, i, node->children[i]);
                if (s.id >= node->keys[i]) {
                    i++;
                }
            }
            insertNonFull(node->children[i], s);
        }
    }

    void displayLevelHelper(BPlusNode* node) const {
        if (!node) return;
        std::queue<BPlusNode*> q;
        q.push(node);
        int level = 0;

        while (!q.empty()) {
            int sz = static_cast<int>(q.size());
            std::cout << " Level " << level++ << ": ";
            for (int k = 0; k < sz; ++k) {
                BPlusNode* curr = q.front();
                q.pop();
                std::cout << "[";
                for (int i = 0; i < curr->numKeys; ++i) {
                    std::cout << curr->keys[i] << (i + 1 < curr->numKeys ? "," : "");
                }
                std::cout << (curr->isLeaf ? "*L" : "") << "] ";

                if (!curr->isLeaf) {
                    for (int i = 0; i <= curr->numKeys; ++i) {
                        if (curr->children[i]) q.push(curr->children[i]);
                    }
                }
            }
            std::cout << "\n";
        }
    }

    void destroy(BPlusNode* node) {
        if (!node) return;
        if (!node->isLeaf) {
            for (int i = 0; i <= node->numKeys; ++i) {
                destroy(node->children[i]);
            }
        }
        delete node;
    }

public:
    BPlusTree() : root(nullptr) {}
    ~BPlusTree() { destroy(root); }

    void insert(const Student& s) {
        if (!root) {
            root = new BPlusNode(true);
            root->keys[0] = s.id;
            root->records[0] = s;
            root->numKeys = 1;
            return;
        }
```

```
        // If root is full, split root and create new root
        if (root->numKeys == (BPLUS_ORDER - 1)) {
            BPlusNode* newRoot = new BPlusNode(false);
            newRoot->children[0] = root;
            splitChild(newRoot, 0, root);
            root = newRoot;
        }
        insertNonFull(root, s);
    }

    // Exact search by ID
    bool search(int id, Student& result) const {
        BPlusNode* leaf = findLeaf(root, id);
        if (!leaf) return false;
        for (int i = 0; i < leaf->numKeys; ++i) {
            if (leaf->keys[i] == id) {
                result = leaf->records[i];
                return true;
            }
        }
        return false;
    }

    // Range query [minId, maxId] using linked leaves
    void rangeQuery(int minId, int maxId) const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] B+ Tree is empty.\n" << Color::RESET;
            return;
        }

        std::cout << "\n" << Color::GREEN << Color::BOLD
                  << "--- Range Query Results for Student IDs in [" << minId << " to " << maxId << "] ---"
                  << Color::RESET << "\n";

        BPlusNode* curr = findLeaf(root, minId);
        int matchCount = 0;

        std::cout << Color::CYAN
                  << "+--------+----------------------+------+--------------+---------+----------------------+\n"
                  << "| ID     | Name                 | Div  | Contact      | Average | Marks (M, P, C, CS) |\n"
                  << "+--------+----------------------+------+--------------+---------+----------------------+\n"
                  << Color::RESET;

        while (curr) {
            for (int i = 0; i < curr->numKeys; ++i) {
                if (curr->keys[i] >= minId && curr->keys[i] <= maxId) {
                    curr->records[i].printRow();
                    matchCount++;
                } else if (curr->keys[i] > maxId) {
                    curr = nullptr; // Stop traversal immediately!
                    break;
                }
            }
            if (curr) curr = curr->next; // Advance to next linked leaf block
        }

        std::cout << Color::CYAN << "+--------+----------------------+------+--------------+---------+----------------------+\n" << Color::RESET;
        std::cout << " Found " << matchCount << " student records within range [" << minId << ", " << maxId << "].\n";
    }

    // Display level by level
    void displayLevels() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] B+ Tree is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "--- B+ Tree Level-by-Level Hierarchical Structure (*L = Leaf Node) ---"
                  << Color::RESET << "\n";
        displayLevelHelper(root);
    }

    // Display all leaf records in sorted order by traversing linked leaves
    void displayLeafSequence() const {
        if (!root) {
            std::cout << Color::YELLOW << " [Notice] B+ Tree is empty.\n" << Color::RESET;
            return;
        }
        std::cout << "\n" << Color::MAGENTA << Color::BOLD
                  << "--- Sequential Leaf-Level Scan (Linked Leaves) ---" << Color::RESET << "\n";

        // Find leftmost leaf
        BPlusNode* curr = root;
        while (!curr->isLeaf) curr = curr->children[0];

        int leafBlock = 1;
        while (curr) {
            std::cout << " Leaf Block #" << leafBlock++ << ": ";
            for (int i = 0; i < curr->numKeys; ++i) {
                std::cout << "[" << curr->keys[i] << ": " << curr->records[i].name << "] ";
            }
            std::cout << (curr->next ? "--> " : "--> [END]\n");
            curr = curr->next;
        }
    }
};

// ============================================================================
// Module 6 Manager & Menu Handler
// ============================================================================
class TreeIndexManager {
private:
    BinarySearchTree bst;
    AVLTree avl;
    BPlusTree bPlus;

public:
    TreeIndexManager() {}

    // Synchronize trees with a list of students
    void populateTrees(const std::vector<Student>& students) {
```

```
        for (const auto& s : students) {
            bst.insert(s);
            avl.insert(s);
            bPlus.insert(s);
        }
    }

    void insertStudentAcrossTrees(const Student& s) {
        bst.insert(s);
        avl.insert(s);
        bPlus.insert(s);
    }

    void deleteStudentAcrossTrees(int id) {
        bst.remove(id);
        avl.remove(id);
    }

    void runMenu(const std::vector<Student>& sharedStudents) {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "═══════════════════════════════════════════════════════\n"
                      << "       MODULE 6: HIERARCHICAL INDEXING USING TREES     \n"
                      << "═══════════════════════════════════════════════════════\n"
                      << Color::RESET;
            std::cout << " [SECTION 1: Binary Search Tree (BST)]\n"
                      << "   1. BST Traversals (Inorder, Preorder, Postorder)\n"
                      << "   2. BST Height & Level-Order Traversal\n"
                      << "   3. Display BST Parent Nodes with Children & Leaf Nodes\n"
                      << "   4. Display BST Mirror Image & Deep Copy Test\n"
                      << "   5. Delete a Student Key from BST\n"
                      << " [SECTION 2: AVL Tree (Self-Balancing)]\n"
                      << "   6. Display AVL Tree (Height & Balance Factors)\n"
                      << "   7. Insert Test Keys Triggering LL, RR, LR, RL Rotations\n"
                      << "   8. Search in AVL Tree\n"
                      << " [SECTION 3: B+ Tree Indexing]\n"
                      << "   9. Display B+ Tree Level-by-Level\n"
                      << "  10. Display B+ Tree Sequential Linked Leaves\n"
                      << "  11. Perform Range Query [MinID to MaxID] on B+ Tree\n"
                      << " [SECTION 4: Comparative Complexity]\n"
                      << "  12. Comprehensive Comparison: BST vs AVL vs B+ Tree\n"
                      << "   0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-12): ");
            if (choice == 0) break;

            switch (choice) {
                case 1:
                    bst.inorder();
                    bst.preorder();
                    bst.postorder();
                    break;
                case 2:
                    std::cout << Color::GREEN << " BST Height: " << bst.getHeight() << Color::RESET << "\n";
                    bst.levelOrder();
                    break;
                case 3:
                    bst.displayParentsAndChildren();
                    bst.displayLeaves();
                    break;
                case 4:
                    bst.displayMirror();
                    bst.verifyDeepCopy();
                    break;
                case 5: {
                    int id = InputHelper::getInteger(" Enter Student ID to delete from BST: ");
                    if (bst.remove(id)) {
                        std::cout << Color::GREEN << " [Success] Node ID " << id << " deleted from BST.\n" << Color::RESET;
                    } else {
                        std::cout << Color::RED << " [Error] ID " << id << " not found in BST.\n" << Color::RESET;
                    }
                    break;
                }
                case 6:
                    avl.displayInorderWithBalance();
                    break;
                case 7: {
                    std::cout << "\n" << Color::CYAN << "--- AVL Rotation Verification Demo ---" << Color::RESET << "\n";
                    AVLTree demoAVL;
                    std::cout << " 1. Demonstrating RR Rotation (Inserting 10, 20, 30)...\n";
                    demoAVL.insert(Student(10, "Demo 10", "A", "00", {}));
                    demoAVL.insert(Student(20, "Demo 20", "A", "00", {}));
                    demoAVL.insert(Student(30, "Demo 30", "A", "00", {}));
                    demoAVL.displayInorderWithBalance();

                    std::cout << " 2. Demonstrating LL Rotation (Inserting 5, 2 into fresh tree)...\n";
                    AVLTree demoLL;
                    demoLL.insert(Student(30, "Demo 30", "A", "00", {}));
                    demoLL.insert(Student(20, "Demo 20", "A", "00", {}));
                    demoLL.insert(Student(10, "Demo 10", "A", "00", {}));
                    demoLL.displayInorderWithBalance();
                    break;
                }
                case 8: {
                    int id = InputHelper::getInteger(" Enter Student ID to search in AVL: ");
                    int comps = 0;
                    AVLNode* node = avl.search(id, comps);
                    if (node) {
                        std::cout << Color::GREEN << " [Found] Student ID " << id << " (" << node->data.name
                                  << ") located in AVL Tree with " << comps << " comparisons.\n" << Color::RESET;
                    } else {
                        std::cout << Color::RED << " [Not Found] Student ID " << id << " not in AVL Tree.\n" << Color::RESET;
                    }
                    break;
                }
                case 9:
                    bPlus.displayLevels();
                    break;
                case 10:
                    bPlus.displayLeafSequence();
                    break;
                case 11: {
```

```
                    int minId = InputHelper::getInteger(" Enter Minimum Student ID: ");
                    int maxId = InputHelper::getInteger(" Enter Maximum Student ID: ");
                    bPlus.rangeQuery(minId, maxId);
                    break;
                }
                case 12: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << "             COMPARATIVE ANALYSIS: BST vs AVL TREE vs B+ TREE                          \n"
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << Color::RESET
                              << " Criterion             | Binary Search Tree (BST) | AVL Tree (Balanced)      | B+ Tree (Multi-way)\n"
                              << "──────────────────────+──────────────────────────+──────────────────────────+──────────────────────\n"
                              << " Search Best Time      | O(1)                     | O(1)                     | O(1)\n"
                              << " Search Worst Time     | O(N) [Skewed Tree]       | O(log2 N) [Guaranteed]   | O(log_M N) [Guaranteed]\n"
                              << " Insertion Worst Time  | O(N)                     | O(log2 N) (1-2 rotations)| O(log_M N) (node splits)\n"
                              << " Deletion Worst Time   | O(N)                     | O(log2 N)                | O(log_M N)\n"
                              << " Tree Height           | Up to N                  | ~ 1.44 log2 N            | log_M N (Very shallow!)\n"
                              << " Range Query Efficiency| Poor (Inorder recursion) | Moderate (O(K + log N))  | EXCELLENT (Linked leaves)\n"
                              << " Disk / Database Fit   | Unsuitable               | High pointer overhead    | INDUSTRY STANDARD\n"
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << Color::CYAN
                              << " [!] KEY INSIGHTS:\n"
                              << "   1. BST degenerates into a singly linked list O(N) on sorted insertions.\n"
                              << "   2. AVL Tree strictly enforces balance factor in {-1, 0, +1}, ensuring O(log N).\n"
                              << "   3. B+ Tree keeps ALL data records in leaves and connects them in a singly linked list.\n"
                              << "      For range queries [A to B], B+ Tree searches once for A, then simply walks the\n"
                              << "      leaf pointers sequentially without re-traversing the tree levels!\n"
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << Color::RESET;
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 12.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE6_TREES_HPP
```

```cpp
#ifndef MODULE7_CAMPUS_GRAPH_HPP
#define MODULE7_CAMPUS_GRAPH_HPP

#include "Common.hpp"
#include <queue>
#include <stack>
#include <vector>

const int INF = 1e9; // Representation for infinity / no direct edge

struct GraphEdge {
    int to;
    int weight;
    GraphEdge* next;
    GraphEdge(int t, int w, GraphEdge* n = nullptr) : to(t), weight(w), next(n) {}
};

class CampusNetwork {
private:
    int numVertices;
    std::vector<std::string> buildingNames;
    std::vector<std::vector<int>> adjMatrix; // Adjacency Matrix representation
    std::vector<GraphEdge*> adjList;         // Adjacency List representation

    void clearAdjList() {
        for (int i = 0; i < numVertices; ++i) {
            GraphEdge* curr = adjList[i];
            while (curr) {
                GraphEdge* temp = curr;
                curr = curr->next;
                delete temp;
            }
            adjList[i] = nullptr;
        }
    }

public:
    CampusNetwork(int vertices = 7) : numVertices(vertices) {
        init(vertices);
        loadDefaultCampusMap();
    }

    ~CampusNetwork() {
        clearAdjList();
    }

    void init(int v) {
        clearAdjList();
        numVertices = v;
        buildingNames.resize(numVertices);
        for (int i = 0; i < numVertices; ++i) {
            buildingNames[i] = "Building " + std::to_string(i);
        }
        adjMatrix.assign(numVertices, std::vector<int>(numVertices, 0));
        adjList.assign(numVertices, nullptr);
    }

    void loadDefaultCampusMap() {
        init(7);
        buildingNames = {
            "Main Gate / Admin",       // 0
            "IT & CS Department",      // 1
            "Central Library",         // 2
            "Central Auditorium",      // 3
            "Student Hostels",         // 4
            "Sports Arena / Ground",   // 5
            "Cafeteria & Student Mess" // 6
        };

        // Undirected weighted roads between campus buildings (distances in meters)
        addEdge(0, 1, 150);
        addEdge(0, 2, 200);
        addEdge(1, 2, 80);
        addEdge(1, 3, 220);
        addEdge(2, 4, 300);
        addEdge(3, 5, 180);
        addEdge(4, 5, 250);
        addEdge(4, 6, 120);
        addEdge(5, 6, 160);
    }

    // Add an undirected weighted edge to both Adjacency Matrix and Adjacency List
    bool addEdge(int u, int v, int weight) {
        if (u < 0 || u >= numVertices || v < 0 || v >= numVertices) {
            std::cout << Color::RED << " [Error] Vertex indices must be between 0 and " << (numVertices - 1) << ".\n" << Color::RESET;
            return false;
        }
        if (weight < 0) {
            std::cout << Color::RED << " [Error] Edge weight (distance) cannot be negative (" << weight << ").\n" << Color::RESET;
            return false;
        }
        if (u == v) {
            std::cout << Color::RED << " [Error] Self loops are not permitted.\n" << Color::RESET;
            return false;
        }

        // 1. Update Adjacency Matrix
        adjMatrix[u][v] = weight;
        adjMatrix[v][u] = weight;

        // 2. Update Adjacency List
        adjList[u] = new GraphEdge(v, weight, adjList[u]);
        adjList[v] = new GraphEdge(u, weight, adjList[v]);

        return true;
    }
```

```
    // Display Adjacency Matrix
    void displayAdjacencyMatrix() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Campus Graph Adjacency Matrix (Distances in Meters) ---" << Color::RESET << "\n";
        std::cout << "      ";
        for (int i = 0; i < numVertices; ++i) {
            std::cout << std::setw(6) << ("[" + std::to_string(i) + "]");
        }
        std::cout << "\n      " << std::string(numVertices * 6, '-') << "\n";

        for (int i = 0; i < numVertices; ++i) {
            std::cout << "[" << i << "] |";
            for (int j = 0; j < numVertices; ++j) {
                if (adjMatrix[i][j] == 0) {
                    std::cout << std::setw(6) << "-";
                } else {
                    std::cout << std::setw(6) << adjMatrix[i][j];
                }
            }
            std::cout << "  (" << buildingNames[i] << ")\n";
        }
    }

    // Display Adjacency List
    void displayAdjacencyList() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD << "--- Campus Graph Adjacency List ---" << Color::RESET << "\n";
        for (int i = 0; i < numVertices; ++i) {
            std::cout << " [" << i << "] " << std::setw(25) << std::left << buildingNames[i] << std::right << " --> ";
            GraphEdge* curr = adjList[i];
            if (!curr) {
                std::cout << "[Isolated / No Roads]";
            }
            while (curr) {
                std::cout << "[" << curr->to << " (" << buildingNames[curr->to] << ") : " << curr->weight << "m] ";
                curr = curr->next;
            }
            std::cout << "\n";
        }
    }

    // 1. Breadth-First Search (BFS) Traversal
    void BFS(int startVertex) const {
        if (startVertex < 0 || startVertex >= numVertices) {
            std::cout << Color::RED << " [Error] Invalid start vertex ID.\n" << Color::RESET;
            return;
        }

        std::vector<bool> visited(numVertices, false);
        std::queue<int> q;

        visited[startVertex] = true;
        q.push(startVertex);

        std::cout << "\n" << Color::GREEN << Color::BOLD
                  << "--- BFS Traversal starting from [" << startVertex << ": " << buildingNames[startVertex] << "] ---"
                  << Color::RESET << "\n Path: ";

        int visitedCount = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            visitedCount++;
            std::cout << "[" << u << ": " << buildingNames[u] << "] ";

            // Traverse neighbors using Adjacency List
            GraphEdge* curr = adjList[u];
            while (curr) {
                if (!visited[curr->to]) {
                    visited[curr->to] = true;
                    q.push(curr->to);
                }
                curr = curr->next;
            }
            if (!q.empty()) std::cout << "-> ";
        }
        std::cout << "\n";

        if (visitedCount < numVertices) {
            std::cout << Color::YELLOW << " [Warning] Graph is DISCONNECTED! Only "
                      << visitedCount << " out of " << numVertices << " buildings were reached.\n" << Color::RESET;
        } else {
            std::cout << Color::GREEN << " [Connected] All campus buildings reached via BFS.\n" << Color::RESET;
        }
    }

    // 2. Depth-First Search (DFS) Traversal
    void dfsHelper(int u, std::vector<bool>& visited, int& count) const {
        visited[u] = true;
        count++;
        std::cout << "[" << u << ": " << buildingNames[u] << "] ";

        // Traverse neighbors
        GraphEdge* curr = adjList[u];
        while (curr) {
            if (!visited[curr->to]) {
                std::cout << "-> ";
                dfsHelper(curr->to, visited, count);
            }
            curr = curr->next;
        }
    }

    void DFS(int startVertex) const {
        if (startVertex < 0 || startVertex >= numVertices) {
            std::cout << Color::RED << " [Error] Invalid start vertex ID.\n" << Color::RESET;
            return;
        }

        std::vector<bool> visited(numVertices, false);
        std::cout << "\n" << Color::MAGENTA << Color::BOLD
                  << "--- DFS Traversal starting from [" << startVertex << ": " << buildingNames[startVertex] << "] ---"
                  << Color::RESET << "\n Path: ";
```

```
        int count = 0;
        dfsHelper(startVertex, visited, count);
        std::cout << "\n";

        if (count < numVertices) {
            std::cout << Color::YELLOW << " [Warning] Disconnected graph! "
                      << (numVertices - count) << " building(s) unreachable from source.\n" << Color::RESET;
        }
    }

    // 3. Minimum Spanning Tree (MST) using Prim's Algorithm
    void primsMST() const {
        std::cout << "\n" << Color::GREEN << Color::BOLD
                  << "--- Prim's Minimum Spanning Tree (Optimal Campus Interconnect) ---" << Color::RESET << "\n";

        std::vector<int> key(numVertices, INF);
        std::vector<int> parent(numVertices, -1);
        std::vector<bool> inMST(numVertices, false);

        // Start from vertex 0
        key[0] = 0;

        for (int count = 0; count < numVertices; ++count) {
            // Find vertex with minimum key value not yet in MST
            int minKey = INF;
            int u = -1;

            for (int v = 0; v < numVertices; ++v) {
                if (!inMST[v] && key[v] < minKey) {
                    minKey = key[v];
                    u = v;
                }
            }

            if (u == -1) break; // Graph is disconnected

            inMST[u] = true;

            // Update keys of adjacent vertices
            GraphEdge* edge = adjList[u];
            while (edge) {
                int v = edge->to;
                int weight = edge->weight;
                if (!inMST[v] && weight < key[v]) {
                    key[v] = weight;
                    parent[v] = u;
                }
                edge = edge->next;
            }
        }

        // Display selected MST edges and compute total cost
        int totalCost = 0;
        int edgeCount = 0;

        std::cout << "+-----------------------------------------+-----------------------------------------+-----------+\n"
                  << "| Building A                              | Building B                              | Distance |\n"
                  << "+-----------------------------------------+-----------------------------------------+-----------+\n";

        for (int i = 1; i < numVertices; ++i) {
            if (parent[i] != -1) {
                totalCost += key[i];
                edgeCount++;
                std::cout << "| " << std::setw(37) << std::left << ("[" + std::to_string(parent[i]) + "] " + buildingNames[parent[i]]) << std::right
                          << " | " << std::setw(37) << std::left << ("[" + std::to_string(i) + "] " + buildingNames[i]) << std::right
                          << " | " << std::setw(6) << key[i] << "m |\n";
            }
        }
        std::cout << "+-----------------------------------------+-----------------------------------------+-----------+\n";

        if (edgeCount < numVertices - 1) {
            std::cout << Color::RED << " [Error] Spanning tree could not be formed because graph is disconnected!\n" << Color::RESET;
        } else {
            std::cout << Color::GREEN << Color::BOLD
                      << " Minimum Total Road / Cabling Cost: " << totalCost << " meters (" << edgeCount << " edges connecting " << numVertices << " buildings)\n"
                      << Color::RESET;
        }
    }

    // 4. Dijkstra's Algorithm for Shortest Path with Full Path Reconstruction
    void dijkstraShortestPath(int src) const {
        if (src < 0 || src >= numVertices) {
            std::cout << Color::RED << " [Error] Invalid source building ID.\n" << Color::RESET;
            return;
        }

        std::vector<int> dist(numVertices, INF);
        std::vector<int> parent(numVertices, -1);
        std::vector<bool> visited(numVertices, false);

        dist[src] = 0;

        for (int count = 0; count < numVertices; ++count) {
            int minDist = INF;
            int u = -1;

            for (int v = 0; v < numVertices; ++v) {
                if (!visited[v] && dist[v] < minDist) {
                    minDist = dist[v];
                    u = v;
                }
            }

            if (u == -1 || dist[u] == INF) break;

            visited[u] = true;

            // Relaxation of neighbors
            GraphEdge* edge = adjList[u];
            while (edge) {
                int v = edge->to;
```

```
                int weight = edge->weight;
                if (!visited[v] && dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    parent[v] = u;
                }
                edge = edge->next;
            }
        }

        // Display results with full reconstructed paths
        std::cout << "\n" << Color::GREEN << Color::BOLD
                  << "--- Dijkstra's Shortest Path from Source: [" << src << ": " << buildingNames[src] << "] ---"
                  << Color::RESET << "\n";
        std::cout << "+----+--------------------------+-----------+-----------------------------------------------------+\n"
                  << "| ID | Destination Building     | Distance | Shortest Route Path Sequence                          |\n"
                  << "+----+--------------------------+-----------+-----------------------------------------------------+\n";

        for (int i = 0; i < numVertices; ++i) {
            std::cout << "| " << std::setw(2) << i
                      << " | " << std::setw(25) << std::left << buildingNames[i] << std::right << " | ";

            if (dist[i] == INF) {
                std::cout << std::setw(8) << "INF" << " | " << Color::RED << std::setw(49) << std::left << "UNREACHABLE (Disconnected)" << std::right << Color::RE
            } else {
                std::cout << std::setw(6) << dist[i] << "m | ";
                // Reconstruct path using parent pointers and stack
                std::stack<int> pathStack;
                int curr = i;
                while (curr != -1) {
                    pathStack.push(curr);
                    curr = parent[curr];
                }

                std::string pathStr = "";
                while (!pathStack.empty()) {
                    int p = pathStack.top();
                    pathStack.pop();
                    pathStr += "[" + std::to_string(p) + "]";
                    if (!pathStack.empty()) pathStr += "->";
                }
                std::cout << std::setw(49) << std::left << pathStr << std::right << " |\n";
            }
        }
        std::cout << "+----+--------------------------+-----------+-----------------------------------------------------+\n";
    }

    // Interactive Menu Handler for Module 7
    void runMenu() {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "=========================================================\n"
                      << "     MODULE 7: CAMPUS NETWORK & GRAPH OPTIMIZATION      \n"
                      << "=========================================================\n"
                      << Color::RESET;
            std::cout << " 1. Display Campus Adjacency Matrix\n"
                      << " 2. Display Campus Adjacency List\n"
                      << " 3. Breadth-First Search (BFS) Traversal\n"
                      << " 4. Depth-First Search (DFS) Traversal\n"
                      << " 5. Prim's Minimum Spanning Tree (MST)\n"
                      << " 6. Dijkstra's Shortest Path from Source Building\n"
                      << " 7. Add Custom Road (Edge) between Buildings\n"
                      << " 8. Reset to Default PCCOE Campus Map\n"
                      << " 9. Algorithmic Complexity Comparison (Matrix vs List & MST vs Dijkstra)\n"
                      << " 0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-9): ");
            if (choice == 0) break;

            switch (choice) {
                case 1:
                    displayAdjacencyMatrix();
                    break;
                case 2:
                    displayAdjacencyList();
                    break;
                case 3: {
                    int src = InputHelper::getInteger(" Enter Starting Building ID (0 to " + std::to_string(numVertices - 1) + "): ");
                    BFS(src);
                    break;
                }
                case 4: {
                    int src = InputHelper::getInteger(" Enter Starting Building ID (0 to " + std::to_string(numVertices - 1) + "): ");
                    DFS(src);
                    break;
                }
                case 5:
                    primsMST();
                    break;
                case 6: {
                    int src = InputHelper::getInteger(" Enter Source Building ID (0 to " + std::to_string(numVertices - 1) + "): ");
                    dijkstraShortestPath(src);
                    break;
                }
                case 7: {
                    int u = InputHelper::getInteger(" Enter First Building ID (0-" + std::to_string(numVertices - 1) + "): ");
                    int v = InputHelper::getInteger(" Enter Second Building ID (0-" + std::to_string(numVertices - 1) + "): ");
                    int w = InputHelper::getInteger(" Enter Road Distance in meters: ");
                    if (addEdge(u, v, w)) {
                        std::cout << Color::GREEN << " [Success] Road added between [" << u << "] and [" << v << "] (" << w << "m).\n" << Color::RESET;
                    }
                    break;
                }
                case 8:
                    loadDefaultCampusMap();
                    std::cout << Color::GREEN << " [Success] Reset to default PCCOE Campus layout (7 buildings, 9 connecting roads).\n" << Color::RESET;
                    break;
                case 9: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "=============================================================================================\n"
                              << "                          CAMPUS GRAPH ALGORITHMIC COMPLEXITY                                \n"
                              << "=============================================================================================\n"
                              << Color::RESET
```

```
                              << " Operation              | Adjacency Matrix | Adjacency List   | Auxiliary Space\n"
                              << "————————————————————————+——————————————————+——————————————————+————————————————————————\n"
                              << " Storage Space          | O(V^2)           | O(V + E)         | Matrix: V^2, List: V+E\n"
                              << " Edge Lookup (u, v)     | O(1)             | O(degree(u))     | O(1)\n"
                              << " BFS Traversal          | O(V^2)           | O(V + E)         | O(V) (Queue)\n"
                              << " DFS Traversal          | O(V^2)           | O(V + E)         | O(V) (Call stack)\n"
                              << " Prim's MST (Array)     | O(V^2)           | O(V^2)           | O(V) (Key, Parent)\n"
                              << " Dijkstra Shortest Path | O(V^2)           | O(E + V log V)*  | O(V) (Dist, Visited)\n"
                              << "————————————————————————+——————————————————+——————————————————+————————————————————————\n"
                              << Color::CYAN
                              << " [!] MST vs SHORTEST PATH TREE (DIJKSTRA):\n"
                              << "   * MST (Prim's): Minimizes the TOTAL SUM of all edge weights required to connect\n"
                              << "     every building together without cycles. Does NOT guarantee shortest path\n"
                              << "     between any specific pair of buildings!\n"
                              << "   * Shortest-Path Tree (Dijkstra): Minimizes the INDIVIDUAL distance from one single\n"
                              << "     source to every other building. Total combined weight of tree is often higher\n"
                              << "     than MST, but optimal for student navigation between two points.\n"
                              << "═══════════════════════════════════════════════════════════════════════════════════════\n"
                              << Color::RESET;
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 9.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE7_CAMPUS_GRAPH_HPP
```

```cpp
#ifndef MODULE8_HASH_TABLE_HPP
#define MODULE8_HASH_TABLE_HPP

#include "Common.hpp"

enum SlotState { EMPTY, OCCUPIED, DELETED };

struct HashSlot {
    int key;
    Student data;
    SlotState state;

    HashSlot() : key(-1), state(EMPTY) {}
};

class StudentHashTable {
private:
    int capacity;
    int occupiedCount;
    std::vector<HashSlot> table;

    // Primary Hash Function: Division Method
    int hashFunction(int key) const {
        return (key % capacity + capacity) % capacity;
    }

public:
    StudentHashTable(int cap = 11) : capacity(cap), occupiedCount(0) {
        table.resize(capacity);
    }

    // Load factor alpha = n / m
    double getLoadFactor() const {
        return static_cast<double>(occupiedCount) / capacity;
    }

    // 1. Insert Student Record with Linear Probing
    // Prevents duplicates. Skips or reuses DELETED slots.
    bool insert(const Student& s, int& probes) {
        probes = 0;
        if (occupiedCount >= capacity) {
            std::cout << Color::RED << " [Error] Hash Table is completely full! Cannot insert ID " << s.id << ".\n" << Color::RESET;
            return false;
        }

        // First check if key already exists to prevent duplicate IDs
        int dummyProbes = 0;
        if (search(s.id, dummyProbes) != -1) {
            std::cout << Color::RED << " [Error] Duplicate Student ID " << s.id << " rejected by Hash Table!\n" << Color::RESET;
            return false;
        }

        int homeIndex = hashFunction(s.id);
        int firstDeletedIndex = -1;

        for (int i = 0; i < capacity; ++i) {
            probes++;
            int idx = (homeIndex + i) % capacity;

            if (table[idx].state == EMPTY) {
                // If we previously found a deleted slot, reuse it!
                int targetIdx = (firstDeletedIndex != -1) ? firstDeletedIndex : idx;
                table[targetIdx].key = s.id;
                table[targetIdx].data = s;
                table[targetIdx].state = OCCUPIED;
                occupiedCount++;
                return true;
            } else if (table[idx].state == DELETED) {
                if (firstDeletedIndex == -1) {
                    firstDeletedIndex = idx; // Remember first tombstone
                }
            }
        }

        // If loop finished, but we found a deleted tombstone
        if (firstDeletedIndex != -1) {
            table[firstDeletedIndex].key = s.id;
            table[firstDeletedIndex].data = s;
            table[firstDeletedIndex].state = OCCUPIED;
            occupiedCount++;
            return true;
        }

        return false;
    }

    // 2. Search for Record by ID using Linear Probing
    // MUST NOT stop at DELETED slots! Probing continues until EMPTY slot or key found.
    int search(int key, int& probes) const {
        probes = 0;
        int homeIndex = hashFunction(key);

        for (int i = 0; i < capacity; ++i) {
            probes++;
            int idx = (homeIndex + i) % capacity;

            if (table[idx].state == EMPTY) {
                // Unoccupied slot encountered -> key definitely not present
                return -1;
            }
            if (table[idx].state == OCCUPIED && table[idx].key == key) {
                return idx; // Found
            }
            // If DELETED, continue probing!
        }
        return -1;
```

```
    }

    // 3. Update Record
    bool update(int key) {
        int probes = 0;
        int idx = search(key, probes);
        if (idx == -1) {
            std::cout << Color::RED << " [Error] Student ID " << key << " not found in Hash Table.\n" << Color::RESET;
            return false;
        }

        Student& s = table[idx].data;
        std::cout << Color::CYAN << " [Updating Student in Hash Table: " << s.name << "]\n" << Color::RESET;
        std::string newContact = InputHelper::getString(" Enter new Contact Number: ");
        if (!newContact.empty()) s.contact = newContact;
        std::string newDiv = InputHelper::getString(" Enter new Division: ");
        if (!newDiv.empty()) s.division = newDiv;

        std::cout << Color::GREEN << " [Success] Hash table record updated successfully.\n" << Color::RESET;
        return true;
    }

    // 4. Delete Record (Marks slot as DELETED / Tombstone)
    bool remove(int key) {
        int probes = 0;
        int idx = search(key, probes);
        if (idx == -1) {
            std::cout << Color::RED << " [Error] Student ID " << key << " does not exist in Hash Table.\n" << Color::RESET;
            return false;
        }

        table[idx].state = DELETED;
        occupiedCount--;
        std::cout << Color::GREEN << " [Success] Student ID " << key << " marked as DELETED (Tombstone set at index "
                  << idx << " to preserve probing chain integrity).\n" << Color::RESET;
        return true;
    }

    // 5. Display Full Hash Table Map
    void display() const {
        std::cout << "\n" << Color::CYAN << Color::BOLD
                  << "--- Open Addressing Hash Table (Linear Probing | Size: " << capacity << ") ---"
                  << Color::RESET << "\n";
        std::cout << "+-------+------------+---------+----------------------+-----------------------+\n"
                  << "| Slot  | Status     | Key/ID  | Home Hash Index      | Student Name          |\n"
                  << "+-------+------------+---------+----------------------+-----------------------+\n";

        for (int i = 0; i < capacity; ++i) {
            std::cout << "| " << std::setw(5) << i << " | ";
            if (table[i].state == OCCUPIED) {
                int home = hashFunction(table[i].key);
                std::string probeDist = (home == i) ? "Home [0]" : ("Probed [+" + std::to_string((i - home + capacity) % capacity) + "]");
                std::cout << Color::GREEN << std::setw(9) << "OCCUPIED" << Color::RESET
                          << " | " << std::setw(7) << table[i].key
                          << " | " << std::setw(18) << (std::to_string(home) + " (" + probeDist + ")")
                          << " | " << std::setw(18) << std::left << table[i].data.name.substr(0, 18) << std::right
                          << " |\n";
            } else if (table[i].state == DELETED) {
                std::cout << Color::MAGENTA << std::setw(9) << "DELETED" << Color::RESET
                          << " | " << std::setw(7) << "-"
                          << " | " << std::setw(18) << "Tombstone Marker"
                          << " | " << std::setw(18) << "-"
                          << " |\n";
            } else {
                std::cout << Color::YELLOW << std::setw(9) << "EMPTY" << Color::RESET
                          << " | " << std::setw(7) << "-"
                          << " | " << std::setw(18) << "-"
                          << " | " << std::setw(18) << "-"
                          << " |\n";
            }
        }
        std::cout << "+-------+------------+---------+----------------------+-----------------------+\n";
        std::cout << " Active Records: " << occupiedCount << " / " << capacity
                  << " | Current Load Factor (alpha): " << std::fixed << std::setprecision(2) << getLoadFactor()
                  << " (" << (getLoadFactor() * 100.0) << "%)\n";
    }

    // Populate hash table from shared student vector
    void populateFromVector(const std::vector<Student>& students) {
        for (const auto& s : students) {
            int probes = 0;
            insert(s, probes);
        }
    }

    // Interactive Menu Handler for Module 8
    void runMenu(const std::vector<Student>& sharedStudents) {
        while (true) {
            std::cout << "\n" << Color::BLUE << Color::BOLD
                      << "============================================================\n"
                      << "      MODULE 8: FAST RECORD RETRIEVAL USING HASHING     \n"
                      << "============================================================\n"
                      << Color::RESET;
            std::cout << " 1. Display Hash Table & Load Factor\n"
                      << " 2. Insert Student into Hash Table\n"
                      << " 3. Search Student by ID (Displays Probes & Time)\n"
                      << " 4. Update Student Record in Hash Table\n"
                      << " 5. Delete Student Record (Demonstrates Tombstones)\n"
                      << " 6. Sync / Rehash from Global Student Database\n"
                      << " 7. Theoretical Analysis: Hash Function, Load Factor & Collisions\n"
                      << " 0. Return to Main Menu\n";

            int choice = InputHelper::getInteger(" Enter your choice (0-7): ");
            if (choice == 0) break;

            switch (choice) {
                case 1:
                    display();
                    break;
                case 2: {
                    int id = InputHelper::getInteger(" Enter Student ID: ");
                    std::string name = InputHelper::getString(" Enter Student Name: ");
```

```
                    std::string div = InputHelper::getString(" Enter Division: ");
                    std::string contact = InputHelper::getString(" Enter Contact: ");
                    Student s(id, name, div, contact, {80, 85, 90, 88});
                    int probes = 0;
                    if (insert(s, probes)) {
                        std::cout << Color::GREEN << " [Success] Inserted ID " << id << " with " << probes << " probe(s).\n" << Color::RESET;
                    }
                    break;
                }
                case 3: {
                    int id = InputHelper::getInteger(" Enter Student ID to search: ");
                    int probes = 0;
                    int idx = search(id, probes);
                    if (idx != -1) {
                        std::cout << Color::GREEN << " [Found] Record located at Slot " << idx
                                  << " with " << probes << " probe(s):\n" << Color::RESET;
                        std::cout << Color::CYAN
                                  << "+--------+----------------------+-------+----------------+---------+----------------------+\n"
                                  << "| ID     | Name                 | Div   | Contact        | Average | Marks (M, P, C, CS) |\n"
                                  << "+--------+----------------------+-------+----------------+---------+----------------------+\n"
                                  << Color::RESET;
                        table[idx].data.printRow();
                        std::cout << Color::CYAN << "+--------+----------------------+-------+----------------+---------+----------------------+\n" << Color::RESET;
                    } else {
                        std::cout << Color::RED << " [Not Found] Student ID " << id << " not present (" << probes << " probes).\n" << Color::RESET;
                    }
                    break;
                }
                case 4: {
                    int id = InputHelper::getInteger(" Enter Student ID to update: ");
                    update(id);
                    break;
                }
                case 5: {
                    int id = InputHelper::getInteger(" Enter Student ID to delete: ");
                    remove(id);
                    break;
                }
                case 6: {
                    populateFromVector(sharedStudents);
                    std::cout << Color::GREEN << " [Synced] Loaded " << sharedStudents.size() << " student records into Hash Table.\n" << Color::RESET;
                    display();
                    break;
                }
                case 7: {
                    std::cout << "\n" << Color::YELLOW << Color::BOLD
                              << "=============================================================================================\n"
                              << "                          HASHING ALGORITHMIC & THEORETICAL ANALYSIS                          \n"
                              << "=============================================================================================\n"
                              << Color::RESET
                              << " 1. HASH FUNCTION (Division Method):\n"
                              << "    h(k) = k mod M (where M = table size, chosen as a prime number like 11 or 13\n"
                              << "    to minimize clustering caused by common divisors in key patterns).\n\n"
                              << " 2. COLLISION RESOLUTION (Linear Probing Open Addressing):\n"
                              << "    h(k, i) = (h(k) + i) mod M for i = 0, 1, 2, ... M-1.\n"
                              << "    - Pro: Excellent cache locality and zero pointer overhead.\n"
                              << "    - Con: Subject to Primary Clustering (blocks of contiguous occupied slots grow,\n"
                              << "      increasing probe times for future collisions).\n\n"
                              << " 3. TOMBSTONES / DELETED SLOTS:\n"
                              << "    - When a slot is cleared, resetting it to EMPTY would break the linear probe\n"
                              << "      chain, causing future searches for colliding elements to falsely report NOT FOUND.\n"
                              << "    - Setting the status to DELETED ensures searches continue probing through the slot,\n"
                              << "      while insertions can freely overwrite it.\n\n"
                              << " 4. LOAD FACTOR (alpha = N / M):\n"
                              << "    - As alpha approaches 1.0 (or >0.7 in open addressing), clustering escalates rapidly.\n"
                              << "    - Average Search Cost ~ (1 + 1 / (1 - alpha)) / 2.\n"
                              << "    - Time Complexity:\n"
                              << "        * Best Case:    O(1) [Zero collisions, direct home slot]\n"
                              << "        * Average Case: O(1) [Assuming alpha <= 0.7]\n"
                              << "        * Worst Case:   O(N) [All keys hash to same slot or table nearly full]\n"
                              << "=============================================================================================\n";
                    break;
                }
                default:
                    std::cout << Color::RED << " Invalid option. Please select 0 to 7.\n" << Color::RESET;
            }
            InputHelper::pause();
        }
    }
};

#endif // MODULE8_HASH_TABLE_HPP
```

```cpp
#ifndef MODULE9_COMPLEXITY_HPP
#define MODULE9_COMPLEXITY_HPP

#include "Common.hpp"

class ComplexityMaster {
public:
    static void displayFullAnalysis() {
        std::cout << "\n" << Color::YELLOW << Color::BOLD
                  << "═══════════════════════════════════════════════════════════════════════════════════════════════════════════════\n"
                  << "               PIMPRI CHINCHWAD COLLEGE OF ENGINEERING, PUNE | S.Y. B.TECH IT 2026-27                          \n"
                  << "               DATA STRUCTURES & ALGORITHMS: MASTER TIME & SPACE COMPLEXITY ANALYSIS                           \n"
                  << "═══════════════════════════════════════════════════════════════════════════════════════════════════════════════\n"
                  << Color::RESET;

        std::cout << Color::CYAN << Color::BOLD
                  << "1. SEARCHING & SORTING ALGORITHMS (MODULES 1 & 2)\n" << Color::RESET
                  << "+----------------------+------------+------------+------------+------------+-----------------------------------------+\n"
                  << "| Algorithm            | Best Time  | Avg Time   | Worst Time | Aux Space  | Key Computational Trait & Factors       |\n"
                  << "+----------------------+------------+------------+------------+------------+-----------------------------------------+\n"
                  << "| Linear Search        | O(1)       | O(N)       | O(N)       | O(1)       | No preconditions; scans sequentially.   |\n"
                  << "| Binary Search        | O(1)       | O(log N)   | O(log N)   | O(1)       | STRICT REQUIREMENT: Input must be sorted.|\n"
                  << "| Bubble Sort          | O(N)       | O(N^2)     | O(N^2)     | O(1)       | Stable; O(N) best case with swapped flag.|\n"
                  << "| Selection Sort       | O(N^2)     | O(N^2)     | O(N^2)     | O(1)       | Unstable; minimizes swaps (at most N-1).|\n"
                  << "| Insertion Sort       | O(N)       | O(N^2)     | O(N^2)     | O(1)       | Stable; adaptive; O(N^2) on reverse input.|\n"
                  << "+----------------------+------------+------------+------------+------------+-----------------------------------------+\n\n";

        std::cout << Color::CYAN << Color::BOLD
                  << "2. LINEAR DATA STRUCTURES (MODULES 4 & 5)\n" << Color::RESET
                  << "+-----------------------------+-----------+-----------+-----------+-----------------------------------------------+\n"
                  << "| Data Structure & Operation  | Time Cmplx| Aux Space  | Discipline| Key Behavioral Characteristic                 |\n"
                  << "+-----------------------------+-----------+-----------+-----------+-----------------------------------------------+\n"
                  << "| Singly Linked List Insert   | O(1) / O(N| O(N)       | Dynamic   | O(1) at head; O(N) at tail without tail ptr.  |\n"
                  << "| Singly Linked List Delete   | O(N)      | O(1)       | Dynamic   | Requires predecessor pointer to update link.  |\n"
                  << "| Doubly Linked List Insert   | O(1)      | O(N)       | Dynamic   | Supports bidirectional traversal (Prev/Next). |\n"
                  << "| Doubly Linked List Delete   | O(1)*     | O(1)       | Dynamic   | O(1) if target node pointer given directly.   |\n"
                  << "| Circular Linked List (CLL)  | O(1)      | O(N)       | Dynamic   | Last node points back to first; cyclic cycle. |\n"
                  << "| Undo Stack (Push / Pop)     | O(1)      | O(N)       | LIFO      | Reverses insertions and deletions safely.     |\n"
                  << "| Infix to Postfix / Prefix   | O(N)      | O(N)       | Operator  | Respects BODMAS precedence & associativity.   |\n"
                  << "| Linear Queue (Enq / Deq)    | O(1)      | O(N)       | FIFO      | Suffers from False Overflow after deqs.       |\n"
                  << "| Circular Queue (Enq / Deq)  | O(1)      | O(N)       | Modulo    | Reuses vacated memory cells via (rear+1)%MAX. |\n"
                  << "+-----------------------------+-----------+-----------+-----------+-----------------------------------------------+\n\n";

        std::cout << Color::CYAN << Color::BOLD
                  << "3. HIERARCHICAL TREE INDEXING (MODULE 6)\n" << Color::RESET
                  << "+--------------------------+------------+------------+------------+------------+-----------------------------------+\n"
                  << "| Tree Structure           | Best Time  | Avg Time   | Worst Time | Aux Space  | Structural Advantage / Limitation |\n"
                  << "+--------------------------+------------+------------+------------+------------+-----------------------------------+\n"
                  << "| Binary Search Tree       | O(1)       | O(log N)   | O(N)       | O(N)       | Degenerates to linked list on sorted.|\n"
                  << "| AVL Tree (Balanced)      | O(1)       | O(log N)   | O(log N)   | O(N)       | Enforces BF in {-1,0,+1}; 4 rotations.|\n"
                  << "| B+ Tree (Order M)        | O(1)       | O(log_M N) | O(log_M N) | O(N)       | Shallow height; linked leaves enable |\n"
                  << "|                          |            |            |            |            | O(log_M N + K) range scan queries!   |\n"
                  << "+--------------------------+------------+------------+------------+------------+-----------------------------------+\n\n";

        std::cout << Color::CYAN << Color::BOLD
                  << "4. GRAPH NETWORK & HASHING (MODULES 7 & 8)\n" << Color::RESET
                  << "+-----------------------------+------------+------------+------------+-----------------------------------------+\n"
                  << "| Algorithm / Structure       | Adj Matrix | Adj List   | Aux Space  | Practical Use in Smart Campus System    |\n"
                  << "+-----------------------------+------------+------------+------------+-----------------------------------------+\n"
                  << "| BFS Traversal               | O(V^2)     | O(V + E)   | O(V)       | Discovers shortest unweighted paths.    |\n"
                  << "| DFS Traversal               | O(V^2)     | O(V + E)   | O(V)       | Cycle detection & campus connectivity.  |\n"
                  << "| Prim's MST Algorithm        | O(V^2)     | O(V^2)     | O(V)       | Minimizes total road/cabling cost.      |\n"
                  << "| Dijkstra Shortest Path      | O(V^2)     | O(V^2)     | O(V)       | Point-to-point shortest navigation route.|\n"
                  << "| Hash Table (Open Address)   | O(1) Avg   | O(N) Worst | O(M)       | Fast O(1) student lookup via ID mod M.  |\n"
                  << "+-----------------------------+------------+------------+------------+-----------------------------------------+\n\n";

        std::cout << Color::GREEN << Color::BOLD
                  << "═══════════════════════════════════════════════════════════════════════════════════════════════════════════════\n"
                  << " SUMMARY OF DESIGN TRADE-OFFS & SELECTION RATIONALE:\n"
                  << "  * Range Queries:       B+ Tree selected over AVL/BST due to contiguous leaf-level linked chains.\n"
                  << "  * Direct ID Lookup:    Hash Table chosen for O(1) average lookup vs O(log N) in balanced trees.\n"
                  << "  * Sparse Grade Data:   Triplet Representation saves memory when matrix density is < 33.3%.\n"
                  << "  * Service Scheduling:  Circular Queue chosen over Linear Queue to prevent false overflow.\n"
                  << "  * Campus Navigation:   Dijkstra's Algorithm used for individual routing; Prim's MST for network infrastructure.\n"
                  << "═══════════════════════════════════════════════════════════════════════════════════════════════════════════════\n"
                  << Color::RESET;
    }
};

#endif // MODULE9_COMPLEXITY_HPP
```

grades.txt — External Raw Input Grade Data File

```
85
72
94
60
88
79
91
55
82
99
48
100
67
74
89
```